

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Природничо-гуманітарний фаховий коледж
ДВНЗ «Ужгородський національний університет»

Реєстраційний номер_____

Дата реєстрації_____

Мотичка Максим Степанович
студент II курсу
групи КН-22
денної форми навчання
ІНП №253108ФМБД

Розробка системи обліку особистих цілей із пріорітизацією
Курсова робота з дисципліни
«Основи програмування та алгоритмічні мови»

Допущено до захисту

Дата «____»_____2026 р.

Підпис _____

Науковий керівник:

Викл. Біла С. В.

Викл. Александра Н. В.

Робота захищена «_____» _____ 2026 р.

з оцінкою «____/____/____»

Підписи членів комісії _____

Ужгород 2026

ЗМІСТ

ВСТУП	4
1 ТЕОРЕТИЧНА ЧАСТИНА	6
1.1 Концептуальні основи та прикладні аспекти розробки	6
1.2 Огляд аналогів	8
1.3 Опис предметної області та постановка задачі	12
1.4 Огляд засобів розробки	15
1.5 Обґрунтування вибору методу розв'язку задачі	18
2 ПРАКТИЧНА ЧАСТИНА	21
2.1 Проєктна частина	21
2.1.1 Інформаційна структура моделі	21
2.1.2 Опис алгоритмічних рішень	24
2.1.3 Функціональні частини програми	27
2.1.4 Фізична структура проєкту	32
2.1.5 Опис структури даних та використаних змінних	38
2.1.6 Тестування проєкту	41
2.2 Експлуатаційна частина	45
2.2.1 Вимоги до технічних засобів	45
2.2.2 Склад інсталяційного пакету та інструкція	47
2.2.3 Опис інтерфейсу	49
2.2.4 Ієрархічна структура проєкту	51
2.2.5 Керівництво користувача та приклади використання програмного продукту	53
ВИСНОВКИ	56
ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ	Error! Bookmark not defined.
Додаток 1. Блок-схема сценарію роботи програми	62
Додаток 2. Блок-схеми алгоритму аналізу тексту	63
Додаток 3. Функціональна схема	64
Додаток 4. Лістинг програми	65

Проходження курсової є невід'ємним елементом фахового становлення майбутніх спеціалістів у галузі програмування. Цей етап дозволяє не лише закріпити здобуту теоретичну базу, а й навчитися застосовувати її для вирішення конкретних прикладних задач. Під час реалізації проєктів особлива увага приділяється опануванню архітектурних стандартів, розробці зручних інтерфейсів та вивченню правил підготовки технічних документів. Робота над програмним продуктом сприяє розвитку навичок об'єктно-орієнтованого моделювання та ефективного керування потоками даних.

Підготовка звіту за результатами практики є обов'язковою частиною навчання, що демонструє рівень професійної зрілості студента. Створення документації вимагає чіткості викладу думок та суворого дотримання стандартів оформлення. У роботі описується логіка побудови алгоритмів, технічні особливості програмних модулів. Такий підхід до фіксації результатів допомагає краще систематизувати отриманий досвід і є важливою підготовкою до написання дипломних робіт.

Актуальність теми зумовлена необхідністю вдосконалення методів організації та контролю виконання особистих цілей у повсякденному житті. У сучасних умовах важливо мати доступні інструменти, що дозволяють швидко фіксувати поставлені цілі, визначати їхню важливість та контролювати поточний стан виконання. Використання можливостей платформи .NET та технології WPF дає змогу створювати надійні настільні додатки, де практичний функціонал обліку поєднується з сучасним дизайном. Розробка засобу для обліку особистих цілей дозволяє на практиці вивчити роботу з колекціями даних та специфіку сортування об'єктів за визначеними пріоритетами.

Процес виконання завдань передбачає детальне вивчення механізмів створення, редагування та систематизації цілей, а також впровадження алгоритмів для визначення їхньої пріоритетності. Документування цих етапів у звіті допомагає розібратися у принципах взаємодії між кодом бізнес-логіки та елементами керування візуального вікна. Оволодіння інструментами версійності коду та принципами побудови UX-інтерфейсів створює міцний фундамент для подальшої кар'єри в IT-сфері. Метою практики є проектування та реалізація WPF-додатку для обліку особистих цілей із можливістю їх пріоритизації, а також належне оформлення результатів у технічному звіті.

Для досягнення мети під час практики заплановано виконання таких завдань: – дослідження теоретичних засад планування особистих цілей та обґрунтування підходу до їх пріоритизації; – побудова структури програмного забезпечення засобами мови C#; – проектування візуальної частини додатку за допомогою декларативної розмітки XAML; – розробка програмних модулів для створення, редагування, сортування та контролю стану цілей; – систематизація етапів розробки та формування фінальної документації.

Об'єктом практики є автоматизація процесів обліку та організації особистих цілей. Предметом дослідження виступає створення прикладного рішення на базі WPF для зберігання, систематизації та пріоритизації цілей і підготовка супровідного звіту. Програма має забезпечувати створення особистих цілей, їх редагування, встановлення пріоритету та відображення поточного стану виконання у наочній формі. Звіт охоплює всі стадії життєвого циклу проєкту: від аналізу вимог до перевірки готового функціоналу.

У ході виконання роботи використовуються принципи системного аналізу та об'єктно-орієнтованого проектування. Ключовим інструментарієм є середовище розробки Visual Studio, мова програмування C# та платформа .NET. Практичне

значення отриманих результатів полягає у створенні функціонального інструменту для обліку та пріоритизації особистих цілей, що відповідає вимогам сучасного користувача. Участь у навчальній практиці сприяє вихованню професійної відповідальності та вмінню грамотно презентувати власні розробки.

1 ТЕОРЕТИЧНА ЧАСТИНА

1.1 Концептуальні основи та прикладні аспекти розробки

Стрімкий розвиток інформаційних технологій сприяє інтеграції різних наукових галузей, що дозволяє створювати комплексні програмні рішення для автоматизації складних процесів. Одним із таких завдань є організація та контроль особистих цілей, що має важливе значення для ефективного планування часу та підвищення особистої продуктивності. Процес розробки подібних систем базується на поєднанні методів інформаційного моделювання, алгоритмізації та системного програмування.

Предметна область роботи охоплює методику обліку особистих цілей для визначення їхньої важливості та контролю стану виконання. Ключовим завданням є систематизація поставлених цілей за пріоритетами, що дозволяє користувачу ефективніше визначати послідовність виконання запланованих завдань. Такі інструменти знаходять широке застосування в особистому плануванні, навчальному процесі та професійній діяльності, де важлива оперативна організація великої кількості поточних справ.

Концепція дослідження передбачає створення цілісного середовища, яке об'єднує обчислювальну логіку та засоби взаємодії з користувачем. Це означає, що програмний продукт має не лише зберігати та обробляти інформацію про поставлені цілі, а й забезпечувати зручне керування цими процесами. Таким чином досягається баланс між технічною складовою (алгоритмами обробки даних) та функціональною частиною (користувацьким досвідом).

Фундаментальним аспектом розробки є попередня організація інформації про особисті цілі. Оскільки користувач може одночасно мати значну кількість запланованих завдань, для отримання зручного представлення необхідно провести їх систематизацію. Це передбачає визначення пріоритету, встановлення поточного

статусу, додавання опису та інших характеристик, що допомагають структурувати інформацію. Подібні операції дозволяють підготувати впорядковані дані для подальшого аналізу та контролю виконання.

Основним елементом бізнес-логіки в межах даної роботи виступає механізм пріоритизації цілей. Такий підхід дозволяє розподіляти поставлені цілі за рівнем важливості, що дає змогу враховувати не лише сам факт їх наявності, а й послідовність виконання. Кількісна оцінка важливості здійснюється за допомогою визначених рівнів пріоритету, наприклад низького, середнього та високого. Це забезпечує зручне впорядкування цілей та дозволяє користувачу зосереджувати увагу на найбільш важливих завданнях.

На рівні програмної реалізації взаємозв'язок теоретичних методів та практичних завдань проявляється у модульній структурі системи. Застосунок має включати підсистеми створення та редагування цілей, обчислювальний модуль для їх сортування та блоки візуалізації даних. Всі ці компоненти функціонують як єдиний механізм, де результати роботи алгоритмів обробки даних перетворюються на зрозумілі показники для кінцевого користувача.

Важливу роль відіграє графічне середовище, яке виступає посередником між людиною та обчислювальним ядром. Проектування інтерфейсу спрямоване на мінімізацію зусиль користувача при виконанні основних операцій: додавання цілей, встановлення їхнього пріоритету, зміни статусу та перегляду поточного списку. Використання сучасних принципів побудови інтерфейсів дозволяє зробити взаємодію з програмою інтуїтивною, що підвищує загальну ефективність інструменту.

Окрім того, значна увага приділяється механізмам обробки та збереження даних. Можливість ведення списку цілей та збереження їхнього поточного стану дозволяє користувачу не лише фіксувати заплановані завдання, а й аналізувати

динаміку їх виконання. Це розширює можливості застосування системи, роблячи її універсальним засобом для особистого планування та контролю поставлених цілей.

Отже, розробка спеціалізованого додатка для обліку особистих цілей із пріоритизацією є прикладом комплексного підходу до вирішення прикладних завдань. Вона демонструє, як поєднання інформаційного моделювання, алгоритмізації та принципів ергономічного дизайну дозволяє створити функціональну та ефективну інформаційну систему.

1.2 Огляд аналогів

У сучасному інформаційному просторі існує розгалужена інфраструктура програмних рішень, призначених для планування та контролю особистих цілей. Ці системи використовуються для організації повсякденних справ, навчальних та професійних завдань, а також для підвищення особистої продуктивності. Кожна з існуючих платформ має власні функціональні особливості, способи організації даних та цільову аудиторію. Нижче наведено детальний аналіз найбільш розповсюджених систем, що виконують функції, споріднені з розроблюваним програмним продуктом.

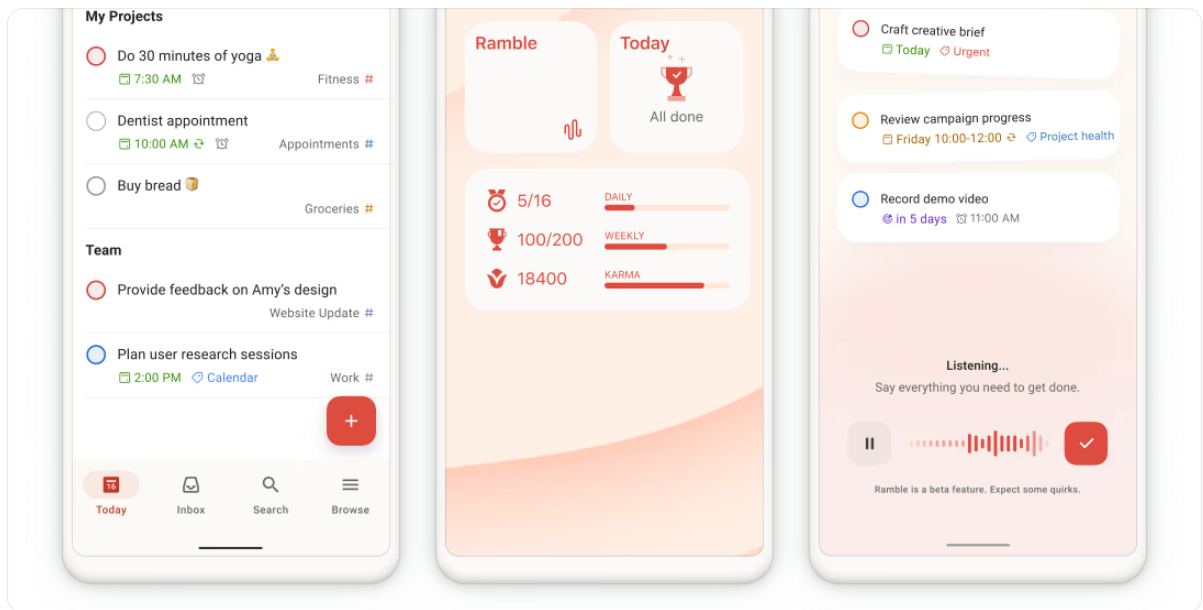


Рисунок 1.1 Вебсайт Todoist

Todoist займає одне з провідних місць серед сучасних систем керування завданнями та особистого планування. Платформа дозволяє створювати структуровані списки завдань, встановлювати терміни виконання, призначати пріоритети та групувати елементи за проектами. Користувач може організовувати власні справи за різними категоріями, що спрощує контроль великої кількості запланованих завдань.

Ключовою перевагою Todoist є широкий набір інструментів для організації робочого процесу та підтримка різних способів представлення інформації. Система дозволяє визначати пріоритетність завдань, що допомагає користувачу зосередитися на найбільш важливих справах. Проте суттєвим недоліком є орієнтація переважно на керування поточними завданнями, а не на довгостроковий облік особистих цілей. Частина розширених функцій також доступна лише за умови використання платної версії.

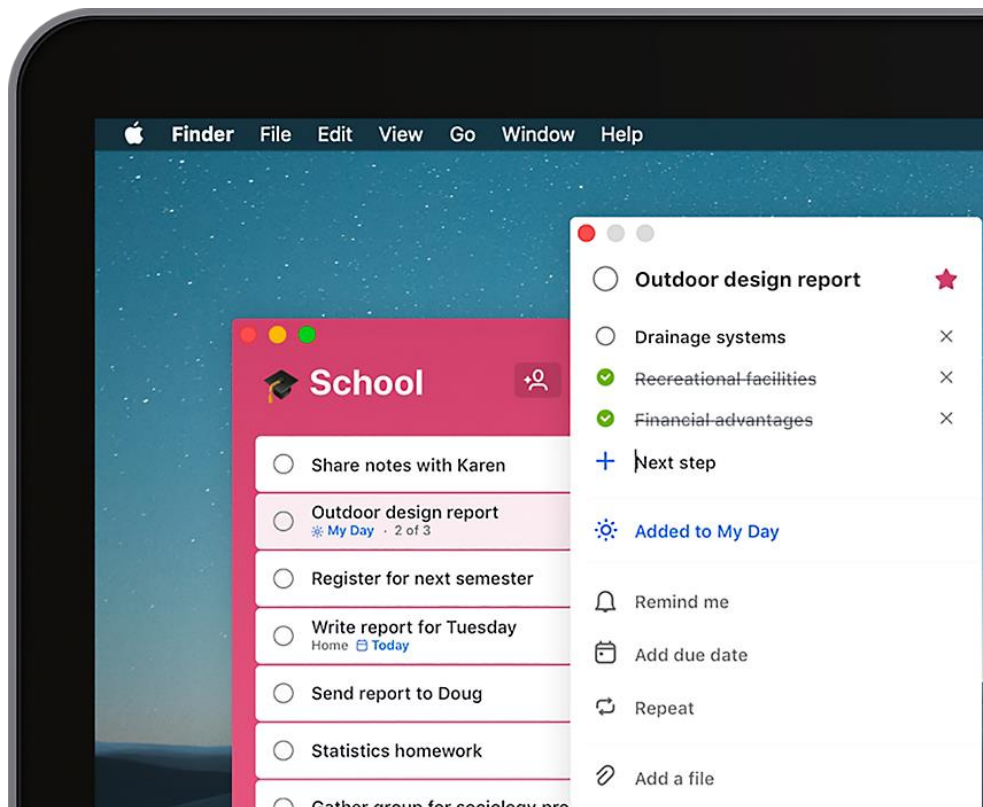


Рисунок 1.2 Вебсайт Microsoft To Do

Microsoft To Do є популярним інструментом для створення та організації особистих списків завдань. Система дозволяє формувати окремі списки, додавати до них завдання, встановлювати терміни виконання та використовувати нагадування. Важливою особливістю платформи є тісна інтеграція з іншими сервісами екосистеми Microsoft, що робить її зручною для користувачів відповідної інфраструктури.

Окрім базового створення завдань, Microsoft To Do надає можливість виділяти важливі елементи та формувати окремий список пріоритетних справ. Це дозволяє частково реалізувати механізм визначення важливості запланованої діяльності. Головним обмеженням системи є відносно простий підхід до пріоритизації та недостатня кількість спеціалізованих інструментів для комплексного аналізу особистих цілей.



Рисунок 1.3 Вебсайт Trello

Trello — це система організації завдань, яка використовує візуальну модель дошок, списків та карток. Такий підхід дозволяє користувачу наочно представляти поточний стан виконання запланованих завдань та переміщувати їх між відповідними етапами. Платформа може використовуватися як для командної роботи, так і для особистого планування.

Система відзначається гнучким налаштуванням робочого простору та зручним візуальним представленням інформації. Користувач може створювати окремі категорії, визначати статуси та доповнювати картки додатковими даними. Проте для простого ведення особистих цілей функціонал Trello може бути надлишковим, а повноцінна організація робочого простору потребує додаткового налаштування. Крім того, основна концепція платформи орієнтована на керування завданнями та проєктами, а не безпосередньо на систему особистих цілей із пріоритизацією.

Окрім професійних платформ, існують також допоміжні інструменти для планування, такі як Google Tasks, Any.do та TickTick. Вони орієнтовані на швидке

створення завдань, встановлення нагадувань та організацію повсякденних справ. Проте такі сервіси зазвичай мають універсальний характер та не завжди дозволяють реалізувати спеціалізований підхід до обліку особистих цілей і визначення їхньої пріоритетності.

Проведений огляд підтверджує, що сучасні ринкові рішення є функціональними, але часто надлишковими для вирішення локальних завдань особистого планування. Більшість з них орієнтована на використання широкого набору додаткових можливостей та постійної синхронізації з хмарними сервісами. Це обґрунтовує доцільність розробки автономного програмного застосунку, який реалізує базові механізми обліку особистих цілей та їх пріоритизації, забезпечуючи простоту, наочність та зручність використання.

1.3 Опис предметної області та постановка задачі

У сучасному інформаційному суспільстві значна частина повсякденних завдань пов'язана з необхідністю планування та контролю власної діяльності. Людина одночасно може мати навчальні, професійні, побутові та особисті цілі, які потребують систематизації та послідовного виконання. У зв'язку зі зростанням кількості завдань виникає проблема їх ефективної організації, оскільки без належного планування важливі цілі можуть залишатися без уваги або виконуватися несвоєчасно. Саме тому виникає необхідність у створенні програмних засобів для автоматизованого обліку, систематизації та пріоритизації особистих цілей.

Предметна область даної роботи охоплює процеси створення, зберігання, редагування та аналізу особистих цілей з метою визначення їхньої важливості та контролю виконання. Основним об'єктом дослідження є цілі користувача, які підлягають обліку та систематизації. У процесі роботи ціль розглядається як структурований набір даних, що дозволяє застосовувати алгоритмічні методи

обробки інформації. Особливістю предметної області є необхідність врахування декількох характеристик цілі, таких як назва, опис, пріоритет, термін виконання та поточний статус. Для ефективної організації даних необхідно забезпечити їх правильне створення, редагування, сортування та відображення.

Актуальність розробки системи обліку особистих цілей із пріоритизацією зумовлена зростанням кількості завдань, які людина повинна контролювати у повсякденному житті. Існуючі універсальні системи планування часто є складними у використанні або містять велику кількість додаткових функцій, які не потрібні для базового обліку особистих цілей. Крім того, частина сучасних систем орієнтована на командну роботу або професійне керування проєктами, тоді як для індивідуального використання часто достатньо локального списку цілей із можливістю визначення їхнього пріоритету. Це створює потребу у розробці простих і доступних програмних засобів, які дозволяють виконувати основні операції з цілями без залежності від складних зовнішніх сервісів.

Метою даної роботи є розробка програмного застосунку для обліку особистих цілей із можливістю визначення та зміни їхньої пріоритетності. Основною задачею є створення інформаційної системи, яка забезпечує автоматизоване додавання, редагування, сортування та контроль стану особистих цілей. Для досягнення поставленої мети необхідно розробити алгоритм додавання цілей у систему, реалізувати механізм їх редагування та видалення, включаючи встановлення основних характеристик, а також реалізувати алгоритм сортування цілей за рівнем пріоритету. Додатково потрібно створити механізм зміни статусу виконання цілей, розробити графічний інтерфейс користувача для зручної взаємодії з системою та забезпечити відображення актуального стану цілей у зрозумілому форматі. Також передбачається можливість збереження даних про створені цілі.

Розроблювана система повинна забезпечувати створення особистих цілей, введення їх назви та опису, встановлення рівня пріоритету, визначення терміну виконання, автоматичне сортування цілей відповідно до їхньої важливості, зміну статусу виконання, відображення інформації у графічному інтерфейсі, а також збереження даних про цілі у файл. Додатково система повинна підтримувати ведення списку створених цілей та забезпечувати можливість швидкого перегляду їхнього поточного стану.

Система повинна відповідати функціональним вимогам, таким як коректне створення та обробка даних про цілі, точне визначення їхнього пріоритету та стабільна робота алгоритмів сортування й обробки інформації. До нефункціональних вимог належать зручний інтуїтивно зрозумілий інтерфейс, швидка обробка даних, можливість роботи без підключення до Інтернету та стабільність роботи програми.

Система має певні обмеження, оскільки зберігання даних здійснюється у форматі JSON-файлів окремо для кожного зареєстрованого користувача, без використання повноцінної зовнішньої бази даних. Передбачено реєстрацію та авторизацію користувачів, що забезпечує розмежування їхніх даних. Синхронізація інформації між декількома пристроями не передбачається, а пріоритизація завдань здійснюється на основі визначених користувачем рівнів важливості.

Таким чином, предметна область даної роботи охоплює процеси обліку, систематизації та пріоритизації особистих цілей на основі алгоритмічних методів. Постановка задачі полягає у створенні програмного застосунку, який дозволяє автоматизувати процес планування особистих цілей, забезпечує зручний інтерфейс користувача та реалізує ефективні механізми їх сортування й контролю виконання. Розробка такої системи є актуальною у зв'язку з необхідністю підвищення

ефективності особистого планування та автоматизації процесів організації повсякденних завдань.

1.4 Огляд засобів розробки

Для реалізації програмного застосунку для обліку особистих цілей із пріоритизацією було обрано платформу .NET та мову програмування C#. Додатково для створення графічного інтерфейсу використано технологію Windows Presentation Foundation (WPF), яка дозволяє розробляти сучасні настільні застосунки з гнучким та інтерактивним інтерфейсом користувача. Вибір даних засобів розробки обумовлений їхньою функціональністю, продуктивністю, широкими можливостями для роботи зі структурованими даними та зручністю використання у навчальних проєктах.

Платформа .NET є сучасним програмним середовищем розробки, яке підтримує багатоплатформеність, високу продуктивність та широкий набір бібліотек для роботи з різними типами даних. Вона надає розробнику готову інфраструктуру для створення настільних, веб- та мобільних застосунків. У контексті даної роботи .NET використовується як основа для виконання бізнес-логіки програми, обробки даних про особисті цілі та реалізації алгоритмів їх сортування і пріоритизації.

Однією з ключових переваг платформи .NET є наявність великої стандартної бібліотеки класів, яка містить інструменти для роботи з файлами, рядками, колекціями, потоками даних та іншими структурами. Це особливо важливо для даного проєкту, оскільки обробка інформації про цілі є основною задачею системи обліку. Завдяки вбудованим можливостям .NET реалізація алгоритмів створення, редагування, сортування та збереження даних про цілі виконується значно простіше та ефективніше.

Мова програмування C# є об'єктно-орієнтованою мовою високого рівня, яка розроблена компанією Microsoft спеціально для платформи .NET. Вона поєднує простоту синтаксису з потужними можливостями для створення складних програмних систем. У даній роботі C# використовується для реалізації всіх основних компонентів застосунку, включаючи бізнес-логіку, алгоритми обробки даних, роботу з файлами та взаємодію з інтерфейсом користувача.

Однією з основних переваг C# є підтримка об'єктно-орієнтованого програмування, що дозволяє структурувати програму у вигляді класів та об'єктів. Це забезпечує модульність, повторне використання коду та простоту супроводу програми. У даному проєкті це проявляється у вигляді окремих класів для роботи з цілями, визначення їхніх пріоритетів, керування статусами, роботи з файлами та управління інтерфейсом.

Окрему роль у розробці відіграє технологія WPF (Windows Presentation Foundation), яка використовується для створення графічного інтерфейсу користувача. WPF є сучасною технологією розробки настільних застосунків для Windows, яка базується на мові розмітки XAML. Вона дозволяє розділяти логіку програми та її візуальне представлення, що значно покращує структурованість коду та спрощує процес розробки інтерфейсу.

Головною перевагою WPF є гнучкість у створенні інтерфейсів. Розробник може легко налаштовувати розміщення елементів, їх стиль, кольорову гаму, анімації та взаємодію з користувачем. У межах даного проєкту WPF використовується для створення інтерфейсу, який включає поля для введення даних про цілі, кнопки для їх створення та редагування, елементи для встановлення пріоритету та статусу, а також блоки для відображення списку та поточного стану цілей.

Мова розмітки XAML, яка використовується у WPF, дозволяє описувати структуру інтерфейсу у декларативному вигляді. Це означає, що візуальна частина програми відокремлюється від логіки, яка реалізується у C#. Такий підхід забезпечує чисту архітектуру застосунку та полегшує його подальше розширення і підтримку.

Для роботи з файлами у проєкті використовуються стандартні засоби .NET, зокрема простір імен System.IO. Він дозволяє здійснювати читання та запис файлів, що є необхідним для збереження інформації про особисті цілі та відновлення даних під час повторного запуску програми. Також стандартні засоби роботи з рядками використовуються для обробки введених користувачем даних та формування інформації для відображення в інтерфейсі.

Важливу роль відіграють колекції, які надаються бібліотекою .NET, зокрема List та інші структури даних. Вони використовуються для зберігання списку особистих цілей та проміжних результатів обробки інформації. Колекції дозволяють ефективно додавати, видаляти, редагувати та сортувати об'єкти відповідно до встановлених користувачем параметрів, зокрема рівня пріоритету та статусу виконання.

Також у проєкті використовується робота з подіями та делегатами C#, що дозволяє реалізувати взаємодію між елементами інтерфейсу та логікою програми. Наприклад, натискання кнопок у WPF викликає відповідні методи у коді, які виконують створення цілей, зміну їхніх параметрів, встановлення пріоритету, зміну статусу або очищення даних.

Таким чином, вибрані засоби розробки забезпечують повну функціональність програмного застосунку, дозволяють ефективно реалізувати механізми обліку та пріоритизації особистих цілей і створити зручний графічний інтерфейс користувача. Поєднання платформи .NET, мови C# та технології WPF є

оптимальним рішенням для реалізації даного навчального проєкту, оскільки забезпечує баланс між простотою розробки, продуктивністю та функціональністю.

1.5 Обґрунтування вибору методу розв'язку задачі

У процесі розробки програмного застосунку для обліку особистих цілей із пріоритизацією ключовим етапом є вибір методу, який дозволяє ефективно визначати ступінь важливості між різними цілями. Враховуючи особливості предметної області, основну увагу було зосереджено на методах структурного ранжування цілей, які не потребують складного математичного моделювання, але при цьому забезпечують достатню наочність результатів. У результаті для реалізації задачі було обрано метод рівнів пріоритету у поєднанні з алгоритмом сортування цілей.

Метод рівнів пріоритету базується на розподілі цілей за визначеними категоріями важливості, де кожна ціль отримує відповідне значення пріоритету. У даній роботі використовується підхід із трьома рівнями, при якому цілі розподіляються на низький, середній та високий пріоритет. Такий підхід дозволяє враховувати не лише сам факт наявності цілі, але й її значущість для користувача, що значно підвищує зручність організації запланованої діяльності.

Основною перевагою методу рівнів пріоритету є його простота та ефективність у реалізації. На відміну від складних математичних моделей, які потребують використання додаткових параметрів або складних алгоритмів оптимізації, даний підхід базується на елементарних операціях обробки даних. Це робить його ідеальним для навчальних проєктів, де важливо продемонструвати принципи роботи алгоритмів без надмірної складності реалізації.

Додатковою перевагою методу є його стійкість до збільшення кількості цілей. Навіть якщо користувач має значну кількість запланованих завдань, розподіл

їх за рівнями важливості дозволяє швидко визначити основні напрямки діяльності. Це особливо важливо в умовах, коли одночасно необхідно контролювати багато різних цілей та визначати послідовність їх виконання.

Для кількісної оцінки важливості цілей у даній роботі використовується числове представлення рівня пріоритету. Такий показник визначається відповідно до встановленої категорії важливості цілі. У контексті даної задачі кожна ціль отримує певне значення, наприклад 1 для низького, 2 для середнього та 3 для високого пріоритету. Використання числового представлення дозволяє автоматизовано сортувати цілі та отримати впорядкований список, що зручно для подальшої роботи користувача.

Важливою причиною вибору саме цього методу є його математична простота та інтерпретованість результатів. Отримане значення легко пов'язується з відповідним рівнем важливості, що дозволяє користувачу швидко оцінити пріоритет кожної цілі. Крім того, метод не потребує складних обчислювальних ресурсів, що робить його придатним для реалізації у настільних застосунках без підключення до серверів або зовнішніх баз даних.

Перед застосуванням алгоритму сортування у програмі виконується етап попередньої обробки даних про цілі. Цей етап включає перевірку введеної інформації, визначення рівня пріоритету та формування структурованого об'єкта цілі. Така підготовка є необхідною для підвищення коректності сортування, оскільки дозволяє уникнути впливу некоректних або неповних даних на порядок відображення цілей.

Після формування структурованого списку цілей виконується їх сортування відповідно до встановлених рівнів пріоритету. Далі система формує впорядкований список, у якому найбільш важливі цілі можуть розміщуватися на початку. На основі отриманих даних користувач може визначити першочергові завдання та

контролювати їх виконання. Чим вищим є значення пріоритету, тим вищим є рівень важливості відповідної цілі.

Альтернативними методами розв'язання задачі могли б бути система числового оцінювання, матриця Ейзенхауера або алгоритми багатокритеріальної оптимізації. Однак ці методи або потребують використання більшої кількості параметрів, або є складнішими для реалізації в межах навчального проєкту. Наприклад, матриця Ейзенхауера потребує одночасного врахування важливості та терміновості, числове оцінювання вимагає детального налаштування шкали, а багатокритеріальні моделі потребують складнішої системи розрахунків.

У порівнянні з цими підходами метод рівнів пріоритету є оптимальним компромісом між простотою реалізації та якістю результатів. Він дозволяє реалізувати базову, але ефективну систему організації особистих цілей, що повністю відповідає вимогам навчального проєкту.

Додатковою перевагою вибраного підходу є його легка інтеграція з графічним інтерфейсом користувача. Результати обробки можуть бути швидко відображені у вигляді списку цілей, позначень рівня пріоритету, індикатора статусу або структурованого переліку, що покращує взаємодію користувача з програмою.

Таким чином, вибір методу рівнів пріоритету у поєднанні з алгоритмом сортування є обґрунтованим з точки зору простоти реалізації, ефективності роботи, зрозумілості результатів та відповідності завданням навчального проєкту. Даний підхід дозволяє створити працездатну систему для обліку та пріоритизації особистих цілей, яка може бути використана як основа для подальшого розширення та вдосконалення функціоналу.

2 ПРАКТИЧНА ЧАСТИНА

2.1 Проєктна частина

2.1.1 Інформаційна структура моделі

Інформаційна складова розробленого програмного продукту побудована на принципах логічного розділення даних користувача, об'єктів особистих цілей та результуючої інформації про їх поточний стан. На відміну від систем, що працюють із зовнішніми базами даних, даний застосунок орієнтований на локальне збереження та обробку інформації про особисті цілі, що забезпечує простоту використання, конфіденційність даних та високу швидкість відгуку.

Основним джерелом постійних даних у системі виступають JSON-файли, у яких зберігаються зареєстровані користувачі та їх особисті цілі. Для роботи з файлами використовується спеціалізований сервіс `JsonStorageService`, інтегрований у програмну логіку застосунку. Це дозволяє виконувати завантаження та збереження структурованих даних без використання зовнішньої бази даних. Дані користувачів зберігаються у файлі `users.json`, а для кожного користувача формується окремий каталог із файлом `goals.json`.

Для представлення основної інформації в пам'яті програми використовується спеціалізований клас-модель `GoalItem`. Він слугує контейнером для зберігання основних характеристик особистої цілі, її пріоритету, стану виконання та дат. Структура організації програмних моделей у проєкті наведена на рисунку 2.1.

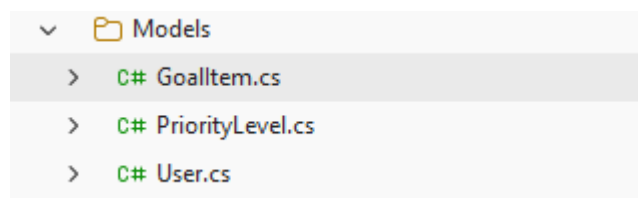


Рисунок 2.1 Розміщення програмної моделі даних у структурі проєкту

Основна інформаційна структура системи представлена об'єктом `GoalItem`, який містить ідентифікатор цілі, ідентифікатор користувача, назву, опис, рівень пріоритету, стан виконання та часові характеристики. Для визначення важливості цілі використовується перелік `PriorityLevel`, який містить три рівні пріоритету: `Low`, `Medium` та `High`. Значення пріоритету використовується під час формування відображення цілей та їх сортування у головному вікні. Детальна специфікація полів об'єкта цілі наведена у таблиці 2.1.

Таблиця 2.1 Структура об'єкта цілі «`GoalItem`»

Назва поля	Тип даних	Призначення поля та спосіб використання
<code>Id</code>	<code>Guid</code>	Унікальний ідентифікатор особистої цілі, який використовується для її однозначного визначення в системі.
<code>UserId</code>	<code>Guid</code>	Ідентифікатор користувача, якому належить ціль. Забезпечує зв'язок цілі з відповідним обліковим записом.
<code>Title</code>	<code>string</code>	Назва особистої цілі, яка коротко описує завдання або бажаний результат.
<code>Description</code>	<code>string</code>	Додатковий текстовий опис цілі, що містить детальнішу інформацію про її зміст.
<code>Priority</code>	<code>PriorityLevel</code>	Рівень пріоритету цілі: <code>Low</code> , <code>Medium</code> або <code>High</code> . Використовується для систематизації та сортування.
<code>IsCompleted</code>	<code>bool</code>	Ознака виконання цілі. Використовується для розділення активних та завершених цілей.

Назва поля	Тип даних	Призначення поля та спосіб використання
CreatedAt	DateTime	Дата та час створення цілі, що дозволяє визначити момент її додавання до системи.
TargetDate	DateTime?	Цільова дата виконання. Використовується для збереження запланованого терміну завершення цілі.

Додатковою інформаційною моделлю системи є клас User, який містить дані облікового запису користувача. До його складу входять ідентифікатор, ім'я користувача, електронна пошта, хеш пароля, сіль та дата створення облікового запису. Для захисту пароля у системі не зберігається його початкове значення. Під час реєстрації формується сіль та обчислюється SHA256-хеш, після чого відповідні значення зберігаються у структурі користувача.

Візуалізація інформації в інтерфейсі забезпечується через систему зв'язків між моделями даних, ViewModel та елементами керування WPF. Основним компонентом відображення є MainWindow, у якому активні цілі користувача розподіляються за рівнями пріоритету. Окремо формується список виконаних цілей, який використовується як архів. Зміна стану цілі, її пріоритету або інших параметрів призводить до оновлення відповідних даних у інтерфейсі. Структура каталогу для локального збереження даних представлена на рисунку 2.2.

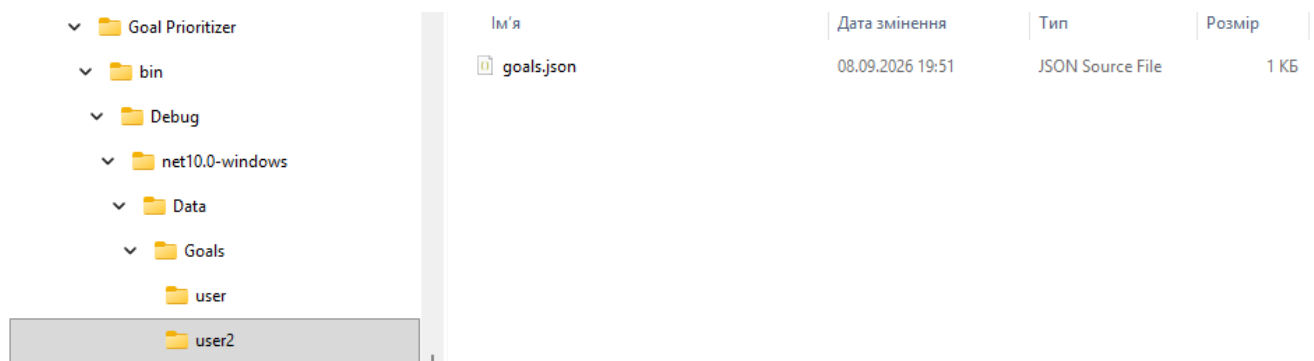


Рисунок 2.2 Структура каталогу збереження даних користувача

Розроблена інформаційна модель базується на принципах цілісності та типізації даних. Використання чітко визначених класів-моделей дозволяє програмі структуровано оперувати інформацією про особисті цілі та користувачів, забезпечуючи коректність збереження, зміни та відображення даних. Такий підхід робить архітектуру даних надійною та придатною для подальшого розширення, наприклад, для додавання нових рівнів пріоритету, додаткових параметрів цілей або розширених механізмів фільтрації.

Після формування структури даних виконується обробка інформації про особисті цілі. Спочатку користувач проходить авторизацію або реєстрацію, після чого система завантажує відповідний список цілей через GoalService. Для кожної цілі зчитуються її назва, опис, рівень пріоритету, стан виконання та цільова дата. Після завантаження дані передаються до MainViewModel, де вони розподіляються за відповідними категоріями та готуються до відображення в інтерфейсі.

Далі виконується систематизація цілей відповідно до їх рівня пріоритету. Кожна ціль належить до одного з трьох рівнів: Low, Medium або High. На основі цього значення цілі розміщуються у відповідних групах та сортуються для зручного перегляду користувачем. Виконані цілі відокремлюються від активних та формують архів. Після внесення змін GoalService передає оновлені дані до JsonStorageService, який серіалізує їх у формат JSON та зберігає у локальному файлі користувача.

2.1.2 Опис алгоритмічних рішень

У цьому розділі наведено опис ключових архітектурних підходів та алгоритмічних рішень, що забезпечують стабільну та швидку роботу системи обліку особистих цілей. Програмний продукт побудований на принципах об'єктно-орієнтованого проєктування (ООП), що гарантує надійність системи та зручність її подальшої підтримки.

Основою побудови інтерфейсу застосунку є подійно-орієнтована архітектура, характерна для платформи WPF. Проєкт розділено на логічні рівні відповідно до патерну MVVM, що дозволяє відокремити візуальну частину від логіки роботи з даними:

Інтерфейсний рівень (View) — представлений XAML-файлами (LoginWindow.xaml, RegisterWindow.xaml, MainWindow.xaml, AddEditGoalWindow.xaml), які описують структуру вікон та елементів керування;

Логічний рівень (ViewModel) — реалізований у файлах C#, де зосереджена логіка взаємодії з інтерфейсом, обробка команд користувача, завантаження та оновлення списків цілей;

Рівень моделей даних (Model) — представлений класами GoalItem, User та PriorityLevel, які структурують інформацію про цілі, користувачів та рівні їх пріоритету для подальшого використання у програмі.

Одним із ключових технічних механізмів є алгоритм завантаження, перевірки та структурування даних про особисті цілі. Після авторизації користувача програма отримує список його цілей через GoalService. Для кожної цілі зчитуються основні характеристики: назва, опис, рівень пріоритету, стан виконання та цільова дата. Отримані дані перевіряються та передаються до MainViewModel для подальшого відображення в інтерфейсі.

Основним алгоритмічним рішенням проєкту є метод пріоритизації та сортування цілей. Кожна ціль отримує один із трьох рівнів пріоритету: Low, Medium або High. Значення PriorityLevel використовується для логічного розподілу цілей та визначення їх порядку відображення. Це дозволяє користувачеві швидко визначити найбільш важливі завдання та зосередити увагу на цілях із високим

пріоритетом. Виконані цілі при цьому відокремлюються від активних та формують архів.

Для оптимізації взаємодії з користувачем застосовано механізм команд та прив'язки даних, передбачений архітектурою MVVM. Клас RelayCommand реалізує інтерфейс ICommand та забезпечує виконання операцій додавання, редагування, видалення та зміни статусу цілі. Клас ViewModelBase реалізує механізм INotifyPropertyChanged, завдяки чому зміни властивостей автоматично передаються до відповідних елементів інтерфейсу без необхідності постійного ручного оновлення вікна.

У контексті проєкту було застосовано наступні принципи та патерни проєктування:

принцип DRY (Don't Repeat Yourself) — повторювані операції роботи з даними винесені в окремі сервіси та методи, що дозволяє використовувати єдину реалізацію для різних операцій із цілями;

принцип Single Responsibility (SRP) — кожен основний клас відповідає за конкретну область функціональності: AuthService відповідає за реєстрацію та авторизацію, GoalService — за управління цілями, JsonStorageService — за локальне збереження та завантаження даних;

патерн MVVM — логіка представлення відокремлена від XAML-інтерфейсу, що спрощує підтримку та подальше розширення програмного продукту;

інкапсуляція даних — моделі GoalItem та User приховують внутрішню структуру інформації та надають типізовані властивості для роботи з нею, що забезпечує цілісність даних у межах програми.

Використання обраних архітектурних рішень та алгоритмів дозволило створити гнучку систему, яка коректно обробляє дані особистих цілей та забезпечує плавну взаємодію з користувачем. Програма працює передбачувано, а її структура відповідає принципам сучасної розробки настільних застосунків на платформі .NET із використанням мови C# та технології WPF.

2.1.3 Функціональні частини програми

Програмний продукт реалізовано на базі платформи Windows Presentation Foundation (WPF) з використанням компонентного підходу та архітектури MVVM. Логічна структура програми побудована на принципах об'єктно-орієнтованого програмування, що забезпечує чітке розмежування процесів візуалізації графічного інтерфейсу, обробки інформації про особисті цілі та логіки взаємодії з файловою системою. Кожен програмний компонент відповідає за конкретну область функціональності системи обліку та пріоритизації особистих цілей. Загальна ієрархія компонентів та організація файлової структури проєкту представлена на рисунку 2.3.

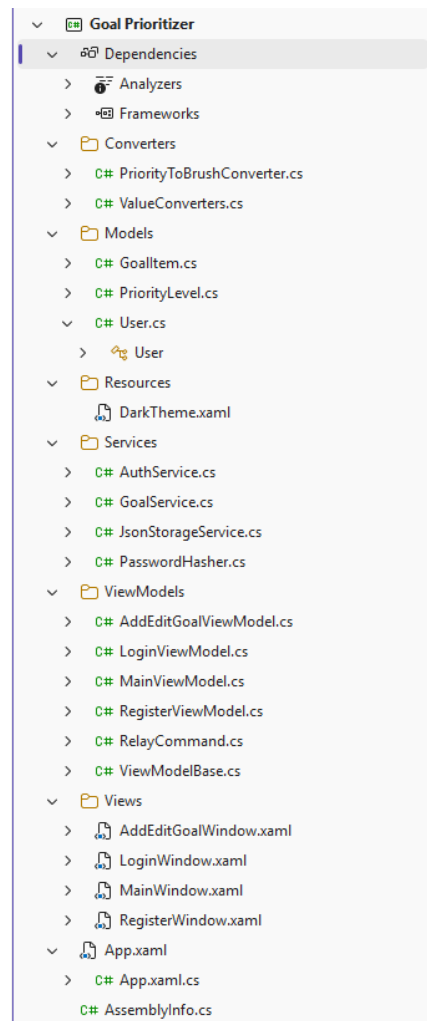


Рисунок 2.3 Ієрархія компонентів та організація файлової структури

Докладний опис та перелік реалізованих функцій і методів представлено на наведених нижче рисунках:

Файл «MainViewModel.cs»:

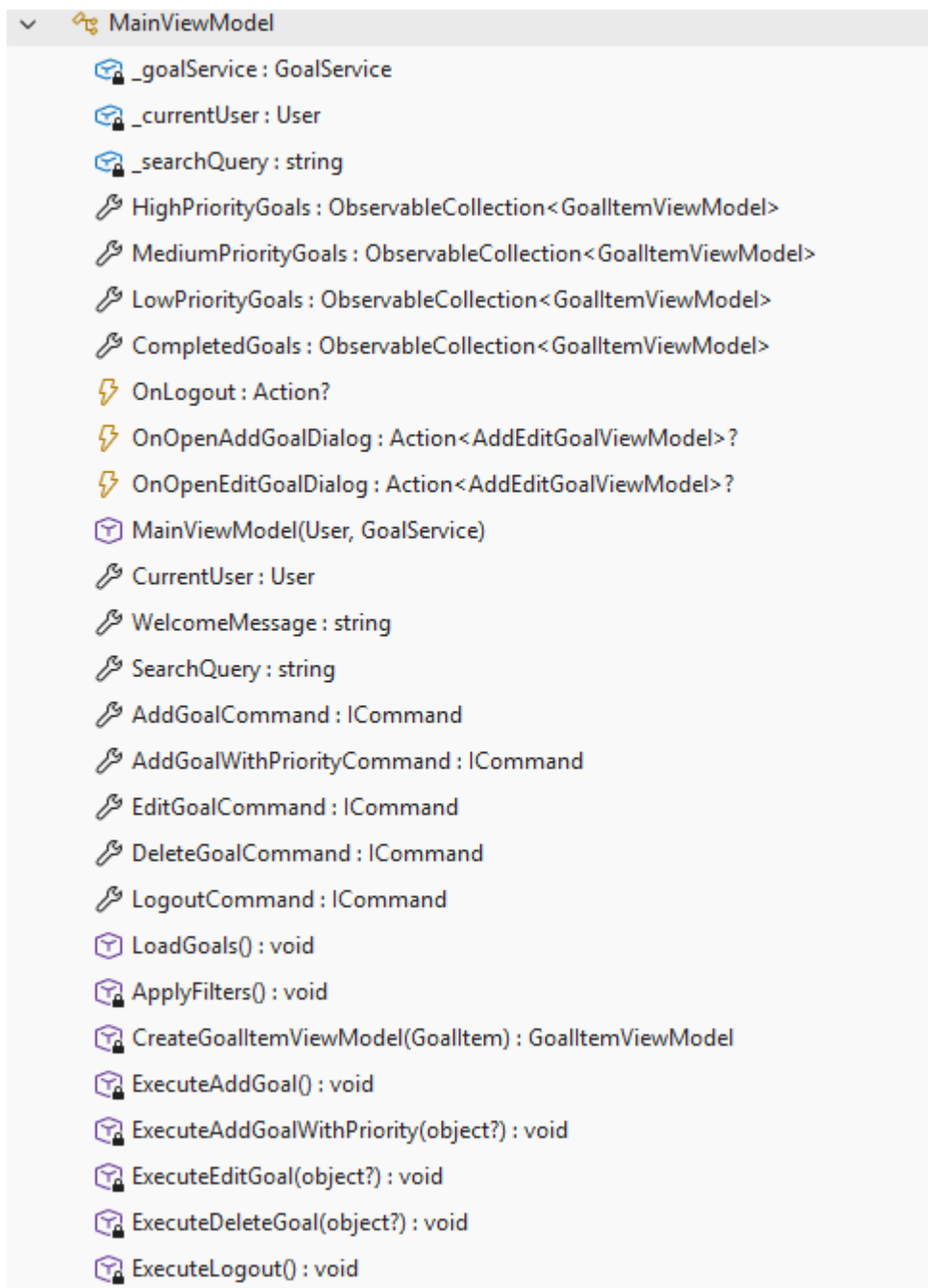


Рисунок 2.4 Структура полів та властивостей класу `MainViewModel.cs`

```

public void LoadGoals()
{
    ApplyFilters();
}

6 references
private void ApplyFilters()
{
    var goals = _goalService.GetGoalsForUser(_currentUser.Username);

    // Фільтрація по пошуковому запиту
    if (!string.IsNullOrEmpty(SearchQuery))
    {
        string query = SearchQuery.Trim().ToLower();
        goals = goals.Where(g =>
            g.Title.ToLower().Contains(query) ||
            g.Description.ToLower().Contains(query)).ToList();
    }

    var activeGoals = goals.Where(g => !g.IsCompleted).ToList();

    // Оновлюємо колонку Високого пріоритету
    HighPriorityGoals.Clear();
    foreach (var goal in activeGoals.Where(g => g.Priority == PriorityLevel.High).OrderBy(g => g.TargetDate ?? DateTime.MaxValue))
    {
        HighPriorityGoals.Add(CreateGoalItemViewModel(goal));
    }

    // Оновлюємо колонку Середнього пріоритету
    MediumPriorityGoals.Clear();
    foreach (var goal in activeGoals.Where(g => g.Priority == PriorityLevel.Medium).OrderBy(g => g.TargetDate ?? DateTime.MaxValue))
    {
        MediumPriorityGoals.Add(CreateGoalItemViewModel(goal));
    }

    // Оновлюємо колонку Низького пріоритету
    LowPriorityGoals.Clear();
    foreach (var goal in activeGoals.Where(g => g.Priority == PriorityLevel.Low).OrderBy(g => g.TargetDate ?? DateTime.MaxValue))
    {
        LowPriorityGoals.Add(CreateGoalItemViewModel(goal));
    }

    // Оновлюємо виконані цілі (Apxia)
    CompletedGoals.Clear();
    var completedItems = goals
        .Where(g => g.IsCompleted)
        .OrderByDescending(g => g.CompletedAt ?? g.CreatedAt);

    foreach (var goal in completedItems)
    {
        CompletedGoals.Add(CreateGoalItemViewModel(goal));
    }
}

```

Рисунок 2.5 Методи завантаження та відображення цілей

```

▼ ViewModels
  ▼ C# AddEditGoalViewModel.cs
    ▼ AddEditGoalViewModel
      _title : string
      _description : string
      _selectedPriority : PriorityLevel
      _targetDate : DateTime?
      _errorMessage : string
      OnSave : Action<GoalItem>?
      OnCancel : Action?
      ExistingGoal : GoalItem?
      AddEditGoalViewModel(GoalItem?)
      WindowTitle : string
      Title : string
      Description : string
      SelectedPriority : PriorityLevel
      TargetDate : DateTime?
      ErrorMessage : string
      Priorities : IEnumerable<PriorityLevel>
      SaveCommand : ICommand
      CancelCommand : ICommand
      ExecuteSave() : void
      ExecuteCancel() : void

```

Рисунок 2.6 Структура та основні методи класу AddEditGoalViewModel

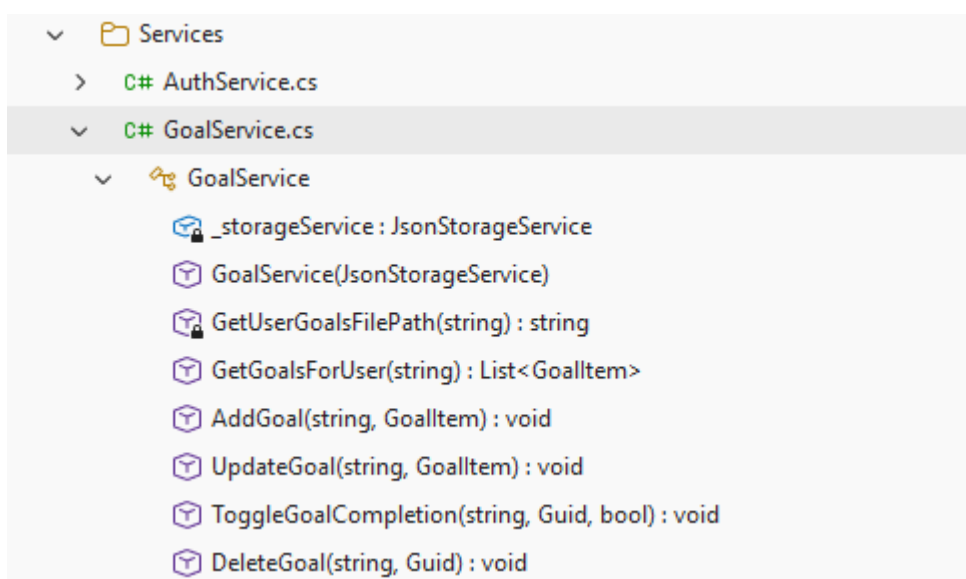


Рисунок 2.7 Структура та основні методи класу GoalService

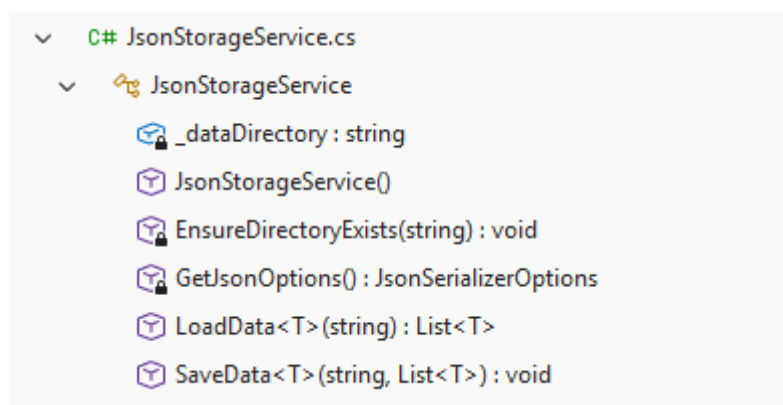


Рисунок 2.11 Методи локального збереження та завантаження даних у форматі JSON

Розроблена функціональна структура програми забезпечує стабільність роботи та коректне опрацювання інформації про особисті цілі. Завдяки використанню компонентного підходу та розподілу функціональності між моделями, представленнями, ViewModel і сервісами програмний код є структурованим та зручним для подальшого розширення. Розподіл обов'язків між елементами інтерфейсу та класами ViewModel дозволяє відокремити логіку

взаємодії з користувачем від операцій роботи з даними. Крім того, використання окремих сервісів для авторизації, управління цілями та локального збереження спрощує тестування і супровід окремих компонентів незалежно від графічної оболонки, що створює надійний фундамент для подальшого розширення функціональності системи обліку та пріоритизації особистих цілей.

2.1.4 Фізична структура проєкту

Фізична структура програмного продукту відображає організацію файлів та каталогів на носії інформації. Проєкт реалізовано на платформі .NET 10 з використанням технології WPF. Файлова система організована за принципом модульності, де вихідний код моделей, представлень, ViewModel, сервісів та інших компонентів рознесений по окремих ієрархічних рівнях.

Кореневий каталог проєкту містить файл рішення (.sln) та папку проєкту, у якій зосереджені основні компоненти системи. На верхньому рівні знаходяться файл App.xaml, файли представлень та файл проєкту .csproj. Вихідний код організовано за окремими каталогами Models, Views, ViewModels, Services та Converters. Такий підхід дозволяє чітко відокремити декларативний опис графічного інтерфейсу, моделі даних, логіку представлення та операції роботи із локальними файлами. Загальний склад кореневого каталогу проєкту представлено на рисунку 2.8.

Ім'я	Дата змінення	Тип	Розмір
bin	08.09.2026 19:47	Папка файлів	
Converters	08.09.2026 19:47	Папка файлів	
Models	08.09.2026 19:47	Папка файлів	
obj	08.09.2026 19:49	Папка файлів	
Resources	08.09.2026 19:47	Папка файлів	
Services	08.09.2026 19:47	Папка файлів	
ViewModels	08.09.2026 19:47	Папка файлів	
Views	08.09.2026 19:47	Папка файлів	
App.xaml	08.09.2026 19:24	Windows.XamlDo...	1 КБ
App.xaml.cs	08.09.2026 19:24	Файл CS	2 КБ
AssemblyInfo.cs	08.09.2026 19:23	Файл CS	1 КБ
Goal Prioritizer.csproj	08.09.2026 19:49	VisualStudio.Laun...	1 КБ
Goal Prioritizer.csproj.user	08.09.2026 19:47	Per-User Project O...	1 КБ
GoalTrackerApp.csproj	08.09.2026 19:23	VisualStudio.Laun...	1 КБ

Рисунок 2.8 Склад ієрархії файлів у кореневому каталозі розробки

Детальна ієрархія вкладених каталогів, що формується середовищем розробки Visual Studio в процесі збірки застосунку, наведена на рисунку 2.9. Ця структура демонструє розподіл вихідних файлів та результатів компіляції за відповідними каталогами.

У каталозі bin/Debug/net10.0-windows зберігаються результати компіляції: виконуваний файл проєкту (.exe), динамічні бібліотеки (.dll) та інші необхідні файли. Каталог obj містить проміжні об'єктні файли та службові файли, які створюються під час процесу побудови WPF-застосунку. Така організація дозволяє відокремити вихідний код від результатів компіляції та забезпечує зручність налагодження і повторної збірки програмного продукту.

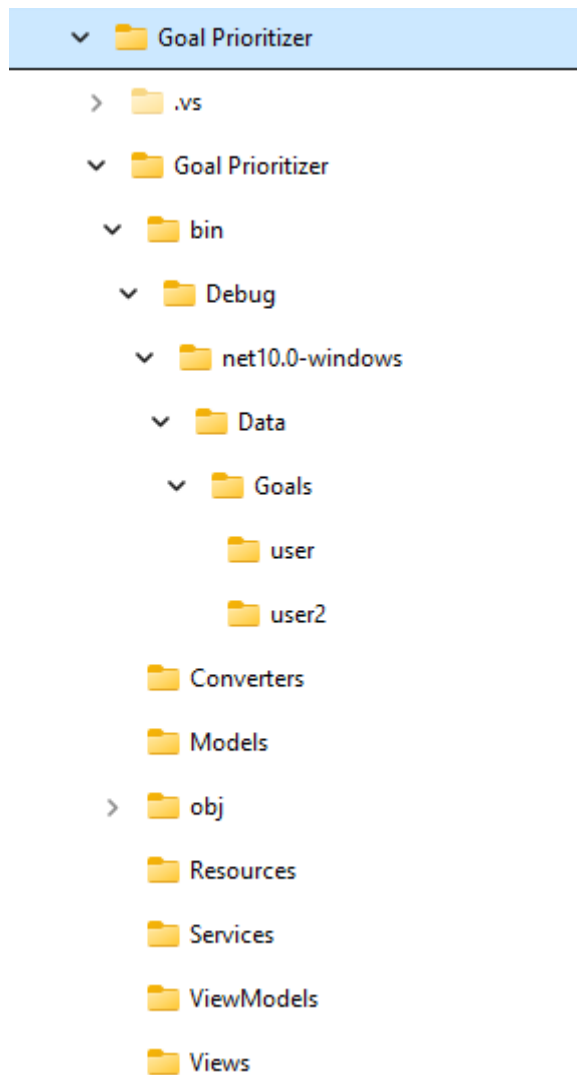


Рисунок 2.9 Архітектура розміщення результатів побудови проєкту

Особливе місце у фізичній структурі займає організація вихідного коду. Оскільки проєкт використовує архітектуру MVVM, програмні компоненти розподілені за функціональним призначенням:

Models — містить моделі даних `GoalItem` та `User`, а також перелік `PriorityLevel`;

Views — містить XAML-файли графічних вікон `LoginWindow`, `RegisterWindow`, `MainWindow` та `AddEditGoalWindow`;

ViewModels — містить класи LoginViewModel, RegisterViewModel, MainViewModel та AddEditGoalViewModel, які реалізують логіку взаємодії інтерфейсу з даними;

Services — містить сервіси AuthService, GoalService, JsonStorageService та PasswordHasher, що відповідають за авторизацію, управління цілями, збереження даних та обробку паролів;

Converters — містить конвертери для перетворення значень моделей у дані, необхідні для відображення в інтерфейсі.

Розміщення основних файлів вихідного коду у структурі проєкту наведено на рисунку 2.10.

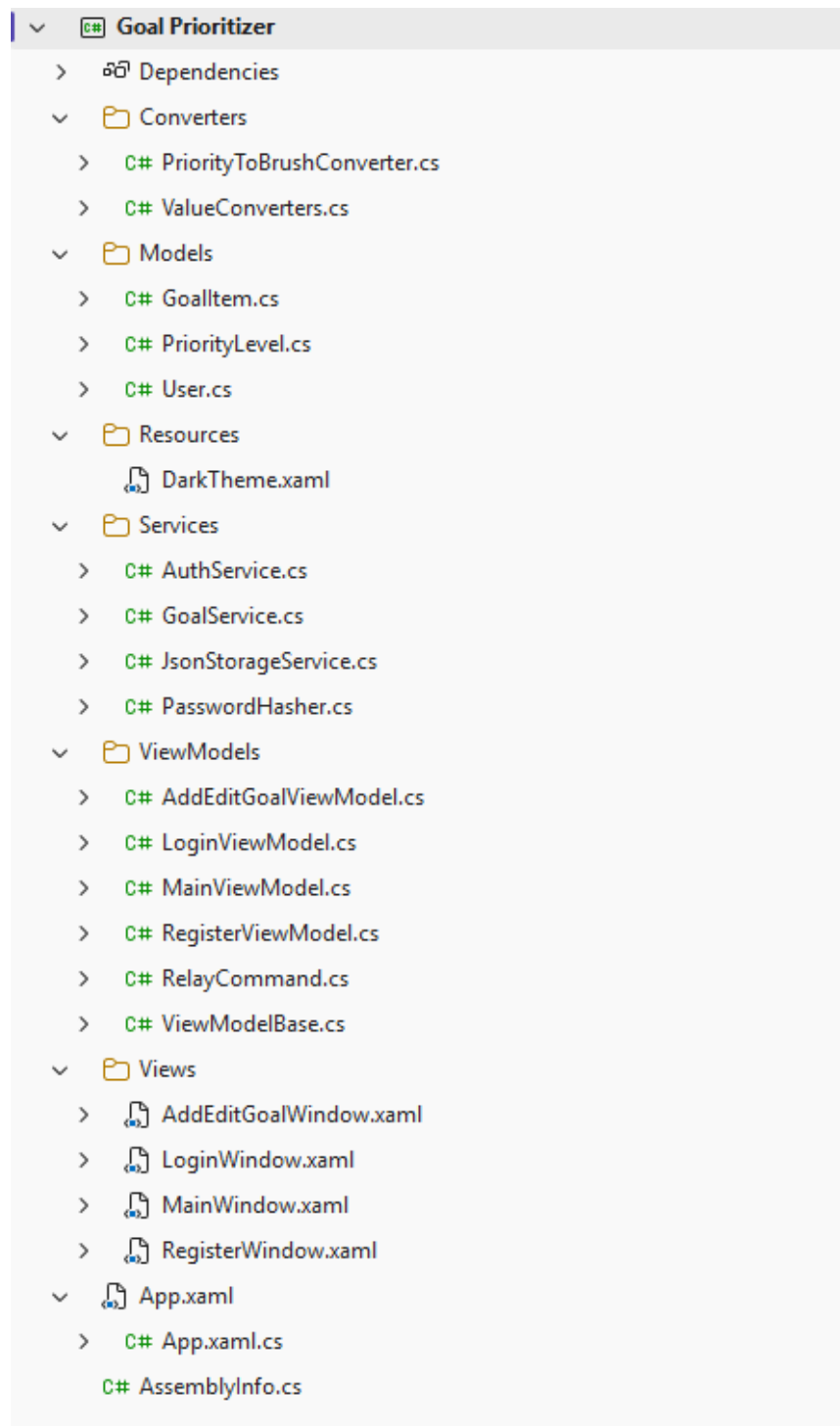


Рисунок 2.10 Фізичне розміщення основних файлів вихідного коду

Окремо від вихідного коду розміщується каталог Data, який використовується програмою для локального збереження інформації. У файлі Data/users.json зберігаються дані зареєстрованих користувачів, а в каталозі

Data/Goals для кожного користувача формується окрема папка із файлом goals.json. Така організація дозволяє ізолювати дані різних користувачів та забезпечує простий доступ до інформації без використання зовнішньої бази даних. Структура каталогу локального збереження даних наведена на рисунку 2.11.

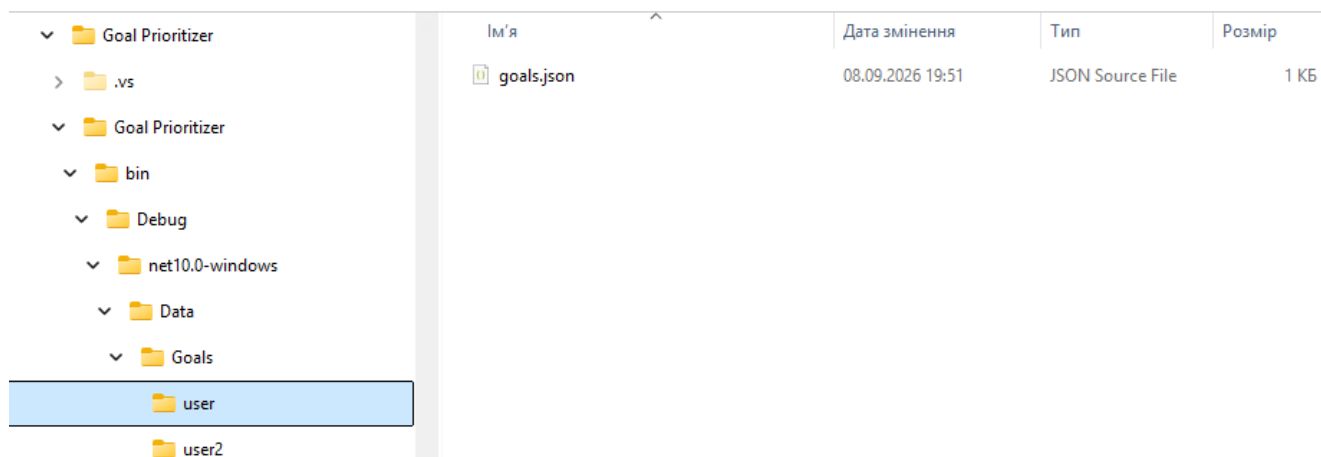


Рисунок 2.11 Структура каталогу локального збереження даних

Розроблена фізична структура проєкту забезпечує чітке логічне розмежування всіх компонентів системи відповідно до їхнього призначення. Організація вихідного коду, службових файлів та локальних даних у спеціалізовані каталоги дозволяє відокремити програмну логіку від інтерфейсних рішень та даних користувачів, що значно спрощує супровід і подальше розширення застосунку.

Така архітектура сприяє дотриманню принципів модульності та забезпечує зручність розгортання програмного продукту як локального настільного застосунку. У підсумку сформована фізична структура створює надійний фундамент для стабільної роботи системи обліку особистих цілей, їх пріоритизації та локального збереження інформації.

2.1.5 Опис структури даних та використаних змінних

В основі програмного забезпечення проєкту лежить об'єктно-орієнтована модель даних, адаптована для зручного зберігання, редагування, пріоритизації та відображення особистих цілей користувача. Організація даних базується на поєднанні моделей предметної області, об'єктів для роботи з користувачами та спеціалізованих сервісів для керування цілями і їх локальним збереженням.

Процес обробки інформації включає використання властивостей об'єктів моделей для зберігання основних характеристик цілі, а також застосування колекцій для завантаження та групування цілей за рівнем пріоритету. Властивості з механізмом доступу `get/set` забезпечують зручну роботу з даними та їх синхронізацію з елементами інтерфейсу. Кожне поле та властивість класу має чітко визначене функціональне призначення, що забезпечує структурованість роботи системи. Детальний опис використаних компонентів наведено в таблицях нижче.

Таблиця 2.2 Параметри та властивості класу `MainViewModel`

Назва	Тип	Призначення
<code>CurrentUser</code>	<code>User</code>	Зберігає дані поточного авторизованого користувача.
<code>Goals</code>	<code>ObservableCollection<GoalItem></code>	Колекція цілей користувача, що відображаються в основному вікні.
<code>HighPriorityGoals</code>	<code>ObservableCollection<GoalItem></code>	Колекція цілей із високим рівнем пріоритету.
<code>MediumPriorityGoals</code>	<code>ObservableCollection<GoalItem></code>	Колекція цілей із середнім рівнем пріоритету.

Назва	Тип	Призначення
LowPriorityGoals	ObservableCollection<GoalItem>	Колекція цілей із низьким рівнем пріоритету.
CompletedGoals	ObservableCollection<GoalItem>	Колекція виконаних цілей, що формує архів.
GoalService	GoalService	Сервіс для завантаження, додавання, редагування та видалення цілей.

Таблиця 2.3 Структура властивостей моделі даних GoalItem.cs

Назва	Тип	Призначення
Id	Guid	Унікальний ідентифікатор цілі.
UserId	Guid	Ідентифікатор користувача, якому належить ціль.
Title	string	Назва особистої цілі.
Description	string	Детальний опис цілі.
Priority	PriorityLevel	Визначає рівень пріоритету цілі.
IsCompleted	bool	Визначає, чи була ціль виконана.
CreatedAt	DateTime	Дата та час створення цілі.
TargetDate	DateTime?	Запланована дата завершення цілі.

Таблиця 2.4 Компоненти управління даними класу GoalService

Назва	Тип	Призначення
GetGoals	метод	Отримує список цілей поточного користувача.
AddGoal	метод	Додає нову ціль до списку користувача.

Назва	Тип	Призначення
UpdateGoal	метод	Оновлює інформацію про існуючу ціль.
ToggleCompletion	метод	Змінює статус виконання цілі.
DeleteGoal	метод	Видаляє вибрану ціль.
JsonStorageService	JsonStorageService	Забезпечує збереження та завантаження даних у форматі JSON.

Таблиця 2.5 Змінні та властивості моделі користувача User.cs

Назва	Тип	Призначення
Id	Guid	Унікальний ідентифікатор користувача.
Username	string	Ім'я користувача для входу до системи.
Email	string	Адреса електронної пошти користувача.
PasswordHash	string	Хеш пароля користувача.
PasswordSalt	string	Сіль, що використовується під час хешування пароля.
CreatedAt	DateTime	Дата та час створення облікового запису.

Під час розробки структури даних проєкту було впроваджено системний підхід до типізації всіх програмних компонентів. Використання моделі GoalItem дозволило відокремити дані особистої цілі від логіки їх обробки та відображення, що відповідає принципам об'єктно-орієнтованої розробки та архітектурному підходу MVVM.

Застосування колекцій цілей у MainViewModel забезпечує зручне групування інформації за рівнями пріоритету, а використання GoalService дозволяє централізовано виконувати операції додавання, редагування, завершення та видалення цілей. Локальне збереження даних за допомогою JsonStorageService забезпечує збереження інформації між запусками програми. Обрана структура

моделей, властивостей і сервісів створює надійну основу для подальшого розширення системи без необхідності зміни її загальної архітектури.

2.1.6 Тестування проєкту

Тестування розробленого програмного застосунку для обліку особистих цілей із пріоритизацією проводилося з метою перевірки коректності роботи всіх функціональних модулів, оцінки стабільності системи та виявлення можливих помилок у процесі обробки даних. Основний підхід до тестування є ручним, оскільки проєкт має навчальний характер і спрямований на глибоке розуміння алгоритмів та логіки роботи програми. Ручне тестування дозволяє детально простежити виконання кожного етапу роботи із цілями та проаналізувати результати роботи системи.

Головною метою тестування є перевірка правильності роботи всіх основних функцій програми, включаючи реєстрацію та авторизацію користувача, створення нових цілей, введення та редагування даних, встановлення рівня пріоритету, зміну статусу виконання, видалення цілей, їх розподіл за пріоритетами, відображення архіву виконаних цілей та локальне збереження інформації у форматі JSON. Також важливим завданням є перевірка стійкості програми до некоректного введення даних та забезпечення її стабільної роботи без аварійного завершення.

Першим етапом тестування є перевірка запуску програми. Програма повинна коректно запускатися без помилок компіляції та виконання, відображати вікно авторизації та ініціалізувати всі необхідні компоненти інтерфейсу. Після запуску користувач повинен мати можливість взаємодіяти з вікном входу до системи, виконати авторизацію або перейти до реєстрації нового облікового запису.

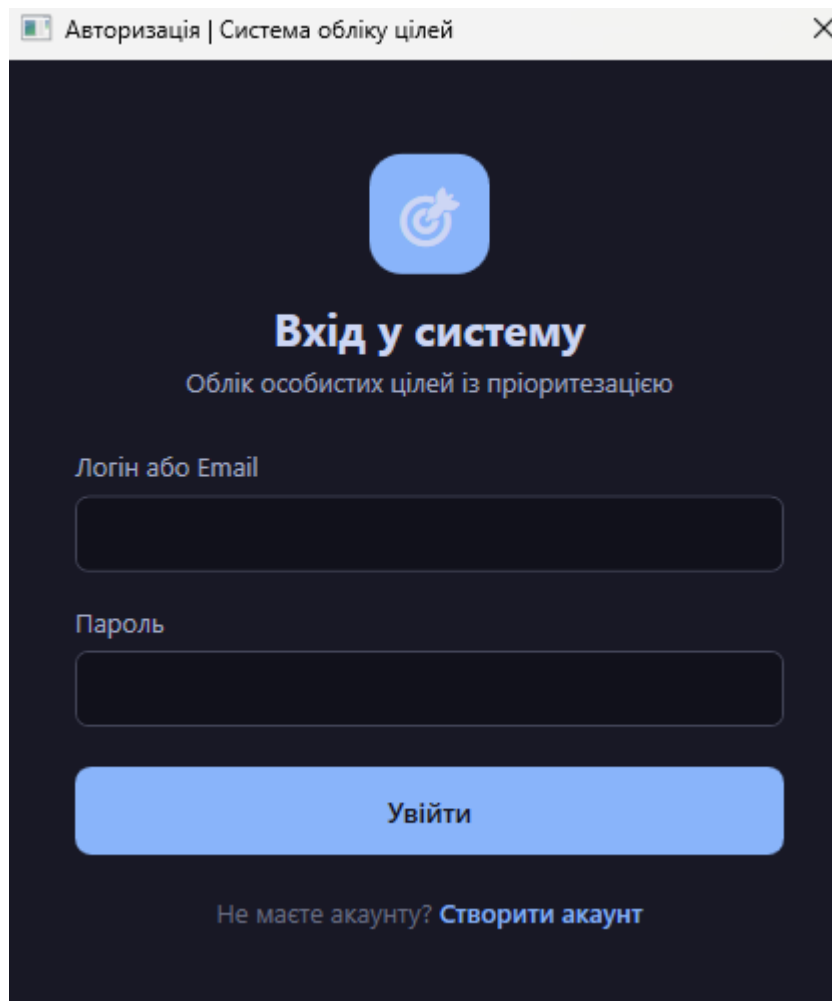


Рисунок 2.1 Запуск програми

Другим етапом є тестування введення та створення даних про особисті цілі. На цьому етапі перевіряється коректність роботи полів введення назви та опису цілі, можливість введення довгих текстових значень, обробка порожніх полів, а також вибір рівня пріоритету та цільової дати. Важливо, щоб програма коректно обробляла введені користувачем дані та передавала їх до відповідних компонентів системи без втрати інформації або виникнення помилок.

Окремо перевіряється поведінка системи при некоректному введенні даних. Наприклад, у ситуаціях, коли користувач не вказує назву цілі або залишає обов'язкові поля порожніми. У таких випадках програма повинна коректно

обробляти помилку та виводити повідомлення про необхідність заповнення відповідних даних, не завершуючи роботу аварійно.

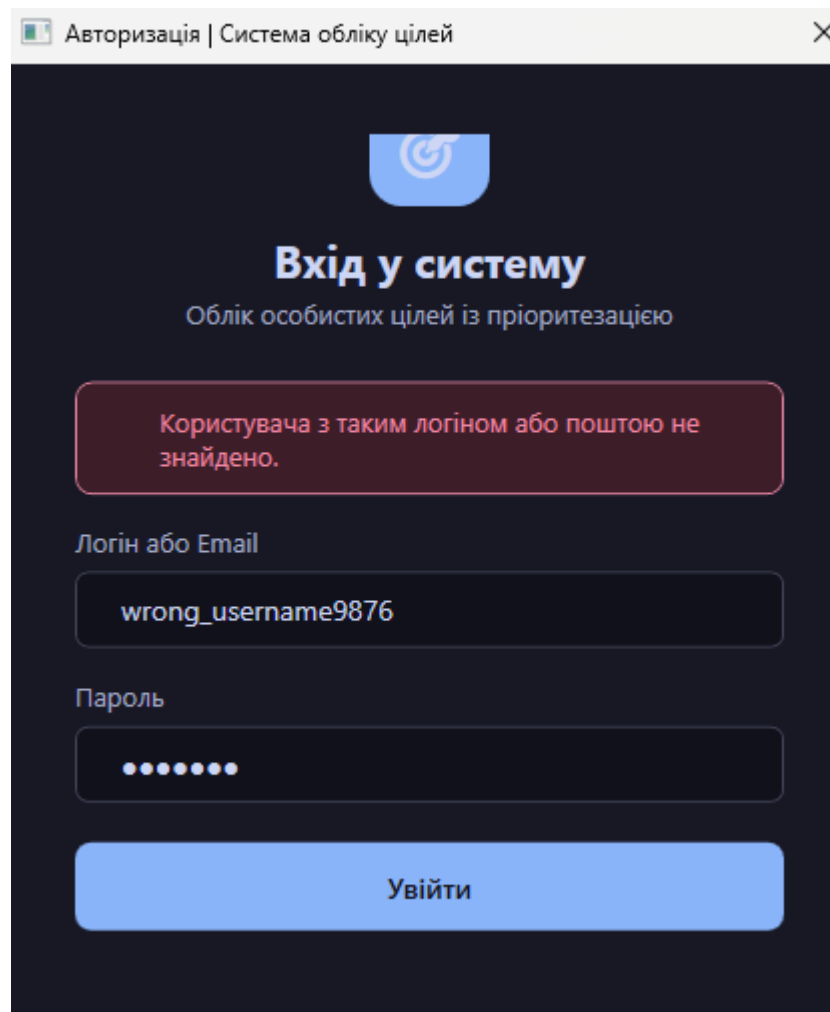


Рисунок 2.2 Некоректне введення даних

Наступним етапом є тестування основних операцій із цілями. Перевіряється можливість додавання нової цілі із зазначенням назви, опису, пріоритету та цільової дати. Після збереження ціль повинна коректно відображатися у відповідній категорії пріоритету.

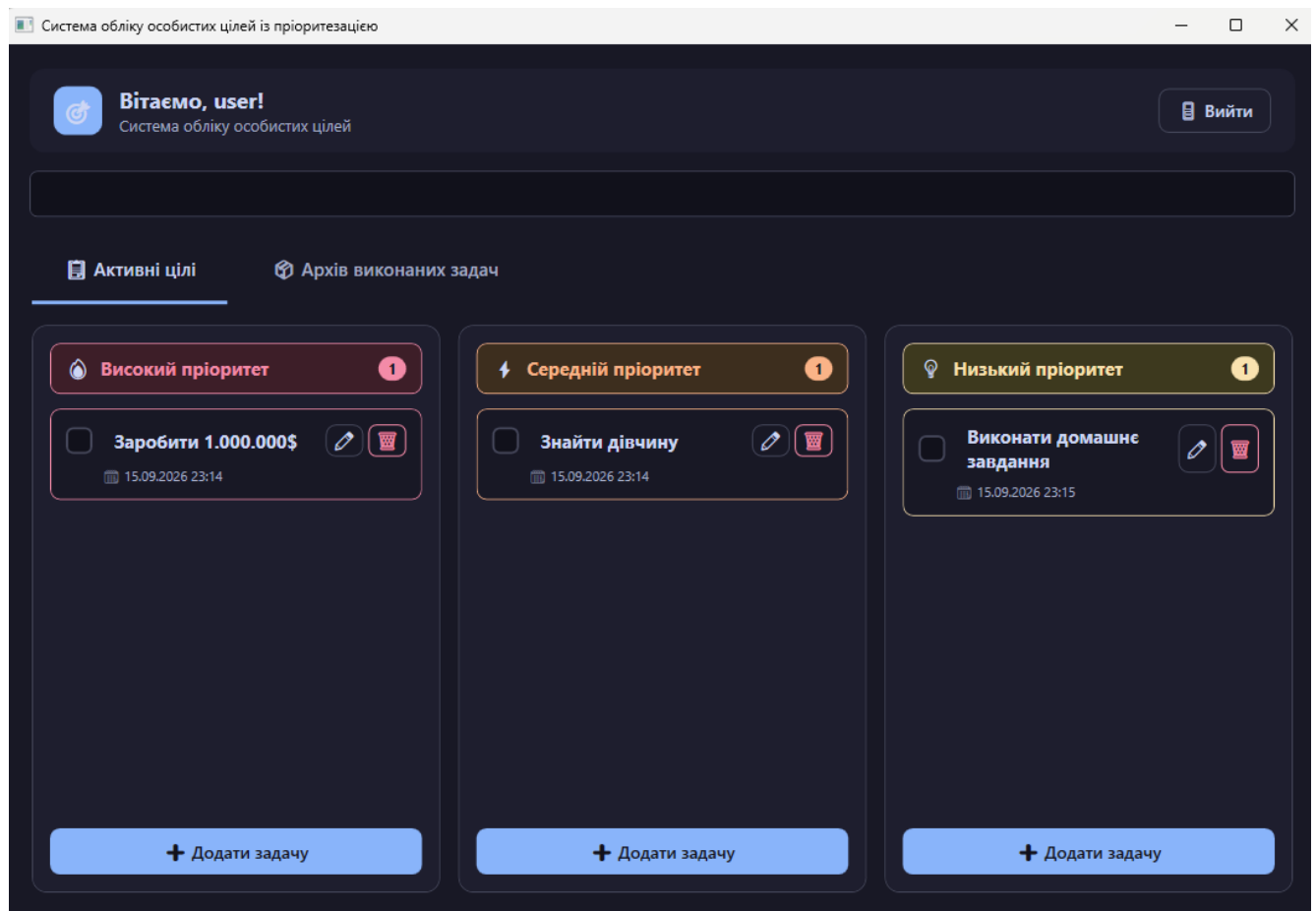


Рисунок 2.12 Результат додавання нових цілей

Далі перевіряється виконання цілі. Після зміни статусу цілі на виконану вона повинна автоматично переміщуватися до архіву виконаних цілей та зберігатися у локальному сховищі.

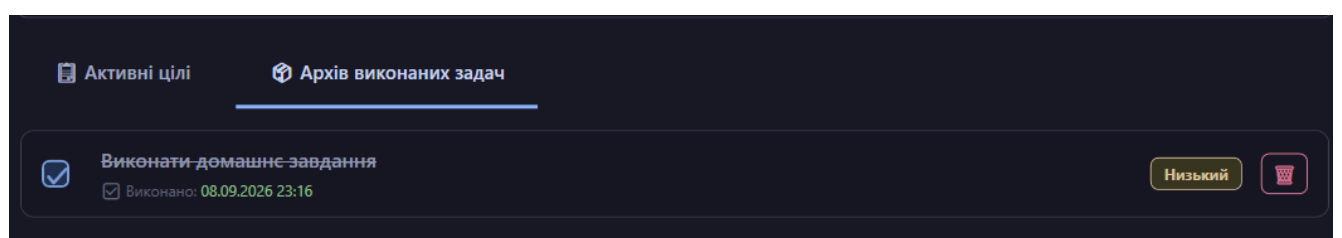


Рисунок 2.13 Результат виконання та переміщення цілі до архіву

Останнім етапом є перевірка видалення цілей. Після виконання команди видалення вибрана ціль повинна зникнути зі списку та бути видалена з локального JSON-сховища.

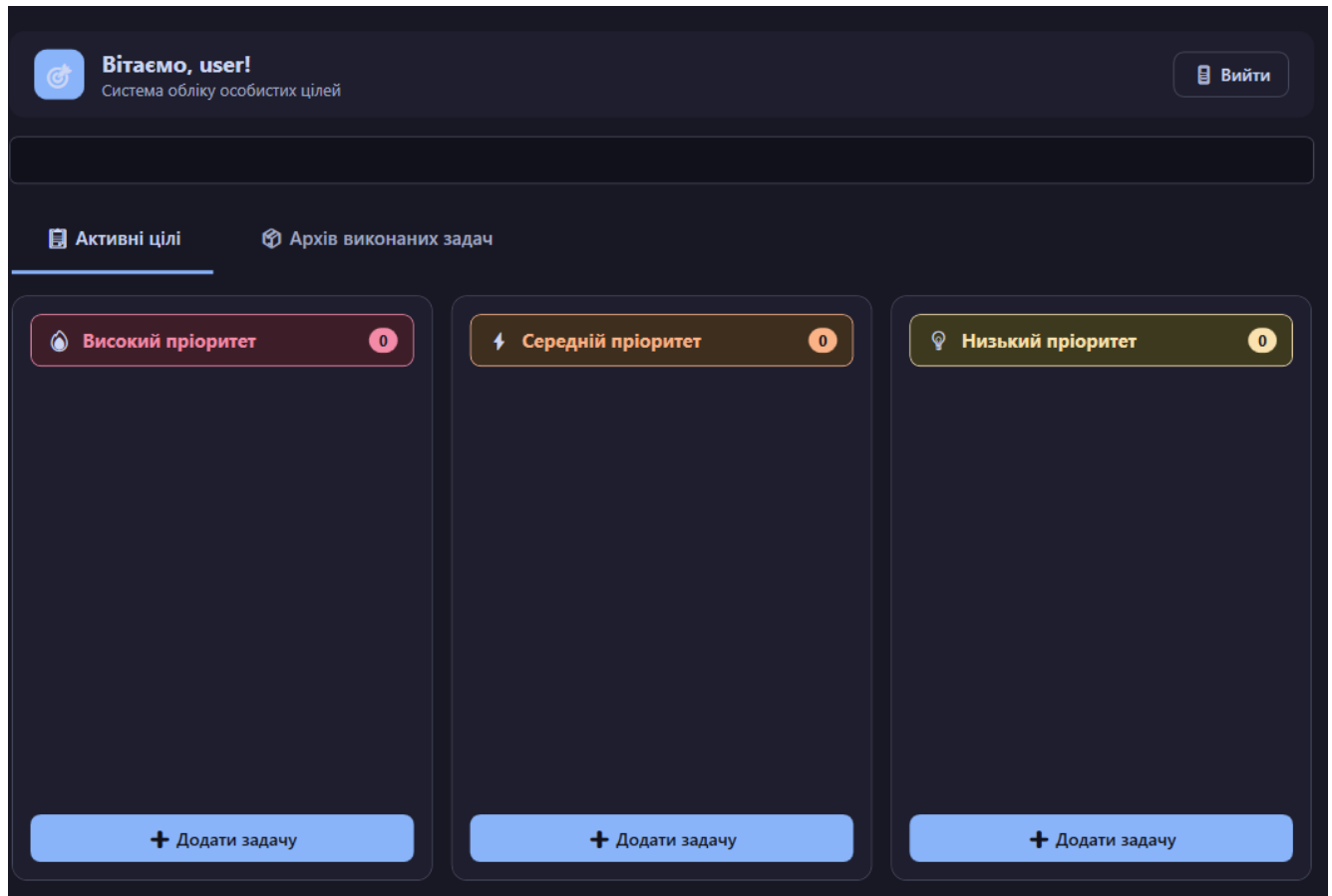


Рисунок 2.14 Результат видалення цілі

Проведене тестування основних функцій підтвердило коректність роботи системи обліку особистих цілей. Додавання, виконання, архівація та видалення цілей працюють відповідно до встановлених вимог, а зміни коректно зберігаються у локальному JSON-сховищі.

2.2 Експлуатаційна частина

2.2.1 Вимоги до технічних засобів

Вимоги до технічних засобів для роботи розробленого програмного застосунку для обліку особистих цілей із пріоритизацією визначають мінімальні та

рекомендовані характеристики апаратного і програмного забезпечення, необхідні для стабільного функціонування системи. Оскільки програма реалізована як настільний застосунок на платформі .NET 10 з використанням технології WPF, вона не потребує високопродуктивного обладнання та може працювати на звичайному персональному комп'ютері або ноутбуці.

До мінімальних вимог належить наявність персонального комп'ютера або ноутбука з операційною системою Windows 10 або Windows 11. Це пов'язано з використанням технології WPF та платформи .NET 10, орієнтованих на роботу в середовищі Windows. Для запуску програми необхідне відповідне середовище виконання .NET 10.

Мінімальні апаратні вимоги включають процесор із тактовою частотою не нижче 1 ГГц та оперативну пам'ять обсягом не менше 2 ГБ. Цих характеристик достатньо для роботи графічного інтерфейсу, обробки списків цілей та виконання операцій із локальними JSON-файлами. При значному збільшенні кількості збережених даних можуть виникати затримки під час їх завантаження на комп'ютерах із низькою продуктивністю.

Для зберігання програмних файлів, бібліотек та локальних даних користувача необхідно мати щонайменше 100 МБ вільного місця на накопичувачі. Цього обсягу достатньо для розміщення виконуваних файлів застосунку та JSON-файлів, у яких зберігаються дані користувачів і їх особисті цілі.

До рекомендованих технічних вимог належать процесор із частотою 2 ГГц і вище, оперативна пам'ять обсягом 4 ГБ або більше та SSD-накопичувач. Такі характеристики забезпечують комфортну роботу застосунку, швидке відкриття вікон та оперативне завантаження локальних даних.

Також рекомендовано використовувати монітор із роздільною здатністю не менше 1366×768 пікселів. Це забезпечує достатній простір для зручного відображення основного вікна застосунку, списків цілей, їх пріоритетів та елементів керування.

Програмні вимоги включають операційну систему Windows 10 або Windows 11 та встановлене середовище виконання .NET 10. Додаткове програмне забезпечення або підключення до зовнішніх серверів не є обов'язковими, оскільки застосунок використовує локальне збереження даних у форматі JSON.

Таким чином, вимоги до технічних засобів є мінімальними та доступними для більшості сучасних персональних комп'ютерів. Розроблений програмний застосунок може працювати без спеціального обладнання, серверної інфраструктури або зовнішньої бази даних, що робить його зручним для локального використання.

2.2.2 Склад інсталяційного пакету та інструкція

Склад інсталяційного пакету розробленого програмного застосунку для обліку особистих цілей із пріоритизацією формується таким чином, щоб забезпечити повну працездатність системи без необхідності складного встановлення додаткових компонентів з боку користувача. Програма реалізована як настільний WPF-застосунок на платформі .NET 10, тому пакет містить необхідні файли для запуску та коректної роботи програмного забезпечення.

Основним елементом пакету є виконуваний файл програми з розширенням .exe, який відповідає за запуск застосунку. Разом із ним до пакету входять необхідні бібліотеки та файли платформи .NET 10, що забезпечують роботу графічного інтерфейсу WPF, моделей, сервісів та інших програмних компонентів системи.

Також до складу пакету входять допоміжні файли, необхідні для коректного запуску застосунку та роботи його компонентів. Окремо створюється каталог Data, у якому зберігаються локальні дані програми у форматі JSON. До нього належить файл users.json, а також каталог Goals, у якому зберігаються файли цілей окремих користувачів.

Застосунок використовує локальне збереження даних, тому для його роботи не потрібне підключення до зовнішньої бази даних або сервера. Дані користувачів та їх цілі зберігаються безпосередньо на комп'ютері, що забезпечує незалежність програми від зовнішньої інфраструктури.

Інструкція з використання програмного застосунку є простою та інтуїтивно зрозумілою. Для початку роботи користувач повинен запустити виконуваний файл програми. Після запуску відкривається вікно авторизації, у якому користувач може виконати вхід до існуючого облікового запису або перейти до реєстрації нового користувача.

Після успішної авторизації відкривається головне вікно програми. У ньому відображаються особисті цілі поточного користувача, розподілені за трьома рівнями пріоритету: високим, середнім та низьким.

Для створення нової цілі користувач відкриває відповідне вікно та вводить її назву, опис, рівень пріоритету та, за необхідності, цільову дату. Після підтвердження дані зберігаються, а нова ціль відображається у відповідній категорії головного вікна.

У процесі роботи користувач може редагувати або видаляти створені цілі. Також передбачена можливість змінити статус цілі на виконану. Після виконання ціль переміщується до архіву виконаних цілей, що дозволяє зберігати інформацію про завершені завдання окремо від активних.

Усі зміни автоматично зберігаються у локальних JSON-файлах. Це забезпечує збереження інформації після завершення роботи програми та дозволяє відновити список цілей під час наступного запуску.

Завершення роботи здійснюється шляхом закриття головного вікна програми або використання стандартних засобів операційної системи.

Таким чином, інсталяційний пакет та інструкція користувача забезпечують простий запуск і використання програмного застосунку без потреби у складних налаштуваннях. Локальне збереження даних та зрозумілий інтерфейс роблять систему зручною для щоденного обліку та пріоритизації особистих цілей.

2.2.3 Опис інтерфейсу

Графічний інтерфейс розробленого програмного застосунку для обліку особистих цілей із пріоритизацією реалізовано за допомогою технології WPF, що забезпечує сучасний вигляд, зручність взаємодії з користувачем та логічне розміщення всіх функціональних елементів. Основною метою при проектуванні інтерфейсу було створення зрозумілої структури, яка дозволяє користувачу швидко виконувати основні операції з особистими цілями без додаткового навчання.

Після запуску програми користувач потрапляє у вікно авторизації. Воно містить поля для введення облікових даних та кнопки для входу до системи або переходу до реєстрації нового користувача. Такий підхід забезпечує розмежування даних різних користувачів та надає доступ до персонального списку цілей після успішної авторизації.

Основне вікно програми призначене для перегляду та керування особистими цілями користувача. Основна частина інтерфейсу поділена на окремі області відповідно до рівня пріоритету цілей. Така структура дозволяє одночасно бачити

цілі різної важливості та швидко визначати завдання, які потребують першочергової уваги.

Окрема область інтерфейсу призначена для цілей із високим пріоритетом. Поруч розташовуються цілі із середнім та низьким рівнями пріоритету. Кожна ціль представлена окремим елементом, який містить основну інформацію, зокрема назву, опис, рівень пріоритету та цільову дату, якщо вона була вказана користувачем.

Для створення нової цілі передбачено окреме вікно додавання та редагування. У ньому користувач може вказати назву та опис цілі, вибрати рівень пріоритету і встановити цільову дату. Після підтвердження введені дані передаються до основного вікна та відображаються у відповідній категорії.

Під час роботи з цілями користувач може редагувати їх дані, змінювати статус виконання або видаляти непотрібні записи. Після позначення цілі як виконаної вона переміщується до окремої області архіву, де зберігається інформація про завершені цілі.

Інтерфейс програми виконаний у темній кольоровій гамі, що надає застосунку сучасного вигляду. Використання контрастного оформлення та візуального розділення категорій пріоритету дозволяє користувачу швидко орієнтуватися у списку цілей та визначати їх важливість.

Таким чином, графічний інтерфейс програмного застосунку є логічно структурованим, інтуїтивно зрозумілим та функціональним. Він забезпечує зручну взаємодію користувача із системою, дозволяє створювати, редагувати, виконувати та видаляти особисті цілі, а також переглядати архів завершених цілей у межах одного застосунку.

2.2.4 Ієрархічна структура проекту

Архітектура розробленого програмного застосунку побудована за принципом розподілу функціональних компонентів, що забезпечує чітке розмежування між візуальним інтерфейсом, логікою роботи програми, моделями даних та системою локального збереження інформації. Загальна функціональна ієрархія компонентів системи, що відображає взаємозв'язок між вікнами програми, моделями, ViewModel та сервісами, наведена на функціональній схемі структури проекту в додатку 3. Така структура дозволяє користувачу послідовно взаємодіяти із системою, переходячи від запуску та авторизації до керування особистими цілями.

На верхньому рівні ієрархії знаходиться програмний комплекс, що об'єднує всі компоненти, необхідні для стабільної роботи системи. Під час запуску файл App.xaml.cs створює необхідні сервіси та відкриває вікно авторизації, після чого користувач отримує доступ до основного функціоналу програми.

Першим функціональним рівнем, з яким взаємодіє користувач, є модуль авторизації. Він забезпечує вхід до існуючого облікового запису та перехід до реєстрації нового користувача. Перевірка введених даних виконується через AuthService, який відповідає за створення та перевірку облікових записів.

Основним рівнем ієрархії є головне вікно програми, де зосереджено основний інструментарій для роботи з особистими цілями. Цей рівень об'єднує декілька взаємопов'язаних компонентів:

модуль керування цілями: забезпечує створення, редагування, виконання та видалення особистих цілей користувача;

система пріоритизації: розподіляє активні цілі за трьома рівнями — високим, середнім та низьким пріоритетом, що дозволяє користувачу визначати першочерговість виконання завдань;

система архівації: відокремлює виконані цілі від активних та відображає їх у відповідній області архіву;

модуль відображення: забезпечує представлення даних про цілі у графічному інтерфейсі головного вікна.

Логіка взаємодії користувача з головним вікном реалізована у класі `MainViewModel`. Він завантажує цілі користувача, розподіляє їх за рівнями пріоритету, формує архів виконаних цілей та обробляє команди додавання, редагування, виконання і видалення.

Окремим структурним елементом системи є рівень роботи з даними. Клас `GoalService` відповідає за виконання операцій із цілями, а `JsonStorageService` забезпечує їх збереження та завантаження у форматі JSON. Дані користувачів зберігаються у файлі `users.json`, а особисті цілі — у відповідних файлах каталогу `Data/Goals`.

Допоміжний рівень ієрархії складається з моделей та компонентів підтримки інтерфейсу. Модель `GoalItem` описує основні властивості особистої цілі, `User` містить дані користувача, а `PriorityLevel` визначає доступні рівні пріоритету. Класи `RelayCommand` та `ViewModelBase` забезпечують взаємодію між графічним інтерфейсом і логікою застосунку відповідно до підходу MVVM.

Повний шлях взаємодії користувача з описаними рівнями системи та послідовність переходів між основними функціональними компонентами представлено на блок-схемі сценарію роботи програми, що міститься у додатку 1.

Таким чином, структура проєкту має логічну послідовність: від ініціалізації програми та авторизації користувача до створення, пріоритизації, виконання та збереження особистих цілей. Такий підхід забезпечує зрозумілу організацію програмного коду, зручність навігації та можливість подальшого розширення функціональності системи.

2.2.5 Керівництво користувача та приклади використання програмного продукту

Керівництво користувача розробленого програмного застосунку для обліку особистих цілей із пріоритизацією призначене для ознайомлення з основними принципами роботи системи, порядком виконання операцій та прикладами використання функціоналу. Програма має інтуїтивно зрозумілий графічний інтерфейс, тому не потребує спеціальної підготовки користувача.

Для початку роботи користувач запускає програму через виконуваний файл. Після запуску відкривається вікно авторизації, у якому необхідно ввести облікові дані та виконати вхід до системи. Якщо користувач ще не має облікового запису, він може перейти до вікна реєстрації та створити новий обліковий запис.

Після успішної авторизації користувач переходить до головного вікна програми, де здійснюється вся подальша робота з особистими цілями. Створені цілі відображаються у відповідних категоріях залежно від встановленого рівня пріоритету.

Для додавання нової цілі користувач відкриває відповідне вікно та вводить її назву й опис. Додатково можна вибрати рівень пріоритету — високий, середній або низький — та встановити цільову дату. Після підтвердження введених даних ціль додається до списку та відображається у відповідній області головного вікна.

Під час роботи користувач може редагувати створені цілі. Для цього необхідно вибрати потрібну ціль та відкрити форму редагування. Після внесення змін та їх підтвердження оновлені дані відображаються у головному вікні програми.

Після досягнення поставленої цілі користувач може змінити її статус на виконаний. Виконана ціль автоматично переміщується до архіву, де зберігається інформація про завершені цілі. Це дозволяє відокремити актуальні цілі від уже виконаних та зберігати історію роботи користувача.

За необхідності користувач може видалити непотрібну ціль. Після виконання відповідної команди ціль видаляється зі списку та локального сховища даних.

Усі створені та змінені дані автоматично зберігаються у локальних JSON-файлах. Завдяки цьому після повторного запуску програми користувач може продовжити роботу зі своїми цілями без необхідності повторного введення інформації.

Приклад використання програми виглядає наступним чином. Користувач авторизується у системі та створює нову ціль, наприклад «Завершити курсову роботу». Для неї встановлюється високий рівень пріоритету та цільова дата. Після збереження ціль відображається у категорії високого пріоритету.

Після виконання поставленого завдання користувач змінює статус цілі на виконаний. Програма переміщує її до архіву виконаних цілей, де вона залишається доступною для перегляду.

Таким чином, програмний застосунок дозволяє користувачу зручно створювати, пріоритизувати, редагувати, виконувати та видаляти особисті цілі, а також зберігати історію виконаних цілей. Інтерфейс є простим та зрозумілим, що

забезпечує зручність роботи навіть для користувачів без спеціальних технічних знань.

ВИСНОВКИ

Виконання курсової роботи з розробки інформаційного середовища з елементами бізнес-логіки для системи обліку особистих цілей із пріоритизацією дозволило комплексно закріпити теоретичні знання та набути практичних навичок у сфері програмування, проектування інтерфейсів користувача та організації роботи з даними. У процесі виконання роботи було створено повноцінний програмний застосунок, який реалізує основні функції створення, редагування, пріоритизації, виконання та збереження особистих цілей користувача.

У ході розробки було досягнуто поставленої мети, яка полягала у створенні зручного графічного інтерфейсу для системи обліку особистих цілей та реалізації програмної частини, що забезпечує їх структуроване зберігання і розподіл за рівнями пріоритету. Програмний продукт дозволяє користувачу створювати нові цілі, задавати їх опис, визначати рівень пріоритету та цільову дату, а також відмічати виконані цілі з подальшим їх переміщенням до архіву.

Одним із важливих результатів виконаної роботи є реалізація логічно структурованого інтерфейсу користувача. Графічна частина програми побудована таким чином, щоб забезпечити максимальну зручність роботи з особистими цілями. Розподіл інтерфейсу на функціональні блоки дозволяє одночасно переглядати цілі різного рівня пріоритету, керувати ними та отримувати доступ до архіву виконаних цілей. Це підвищує ефективність використання програми та робить її інтуїтивно зрозумілою навіть для користувачів без спеціальної підготовки.

У процесі виконання роботи було реалізовано логіку обробки даних про цілі, яка включає створення, редагування, видалення та зміну їхнього стану. Даний етап є важливим, оскільки дозволяє користувачу підтримувати актуальний

перелік власних цілей, своєчасно змінювати необхідні параметри та контролювати процес їх виконання.

Наступним важливим етапом стала реалізація механізму пріоритизації цілей, який дозволяє розподіляти їх за трьома рівнями: високим, середнім та низьким. Саме на основі цього підходу формується структуроване представлення цілей у головному вікні програми. Використання рівнів пріоритету дає змогу користувачу швидко визначати важливість окремих завдань та зосереджувати увагу на найбільш актуальних із них.

Для організації роботи з цілями було реалізовано окремий сервіс GoalService, який відповідає за отримання, додавання, редагування, видалення та зміну стану цілей. Такий підхід дозволив відокремити основну бізнес-логіку від графічного інтерфейсу. Це спрощує структуру програмного коду та забезпечує можливість подальшого розширення функціональності застосунку.

Під час реалізації програмного продукту було використано мову програмування C# та технологію WPF, що забезпечило створення сучасного графічного інтерфейсу. Використання платформи .NET 10 дозволило застосувати вбудовані засоби роботи з файлами, об'єктами та колекціями даних, що значно спростило реалізацію функціональних модулів програми. Крім того, застосування об'єктно-орієнтованого підходу та архітектури MVVM дало змогу структурувати код та розділити логіку програми на окремі компоненти.

Важливим результатом роботи є також реалізація модуля локального збереження даних. Даний функціонал забезпечується класом JsonStorageService та дозволяє зберігати інформацію про користувачів і їхні цілі у JSON-файлах. Для кожного користувача формується окремий файл з даними про цілі, що забезпечує зручну організацію локального сховища та можливість відновлення інформації після повторного запуску програми.

У процесі тестування програмного забезпечення було перевірено коректність роботи всіх основних функцій. Зокрема, було протестовано додавання нових цілей, зміну їхніх параметрів, виконання цілей із переміщенням до архіву, видалення цілей, а також їх збереження у локальних JSON-файлах. Тестування показало, що програма стабільно виконує всі передбачені функції та коректно оновлює дані користувача.

Окрему увагу було приділено перевірці обробки некоректного введення. Програма здійснює перевірку необхідних даних під час реєстрації, авторизації та створення цілей, не допускаючи збереження некоректної інформації. Це свідчить про достатній рівень надійності реалізованої системи та коректну організацію роботи з даними.

У результаті виконання курсової роботи було отримано програмний продукт, який виконує функції системи обліку особистих цілей із можливістю їх пріоритизації. Незважаючи на те, що система має навчальний характер, вона реалізує основні принципи роботи подібних програмних засобів, що можуть використовуватися для організації особистих завдань і контролю їх виконання.

Отримані результати підтверджують правильність обраного підходу до розробки системи, а саме використання архітектури MVVM, об'єктно-орієнтованого підходу та розподілу цілей за рівнями пріоритету. Даний підхід є ефективним для побудови локальної системи керування особистими цілями та може бути використаний як основа для подальшого вдосконалення програмного продукту.

Перспективи розвитку програмного продукту полягають у можливості розширення функціоналу, зокрема впровадження додаткових можливостей фільтрації та сортування цілей, розширення параметрів цілей, удосконалення

роботи з датами та нагадуваннями, а також покращення засобів представлення статистики виконаних цілей.

Також можливим напрямом вдосконалення є розширення можливостей системи керування цілями та архівом виконаних завдань. У подальшому доцільним може бути вдосконалення інтерфейсу користувача, додавання додаткових засобів візуалізації прогресу та реалізація нових механізмів організації особистих цілей.

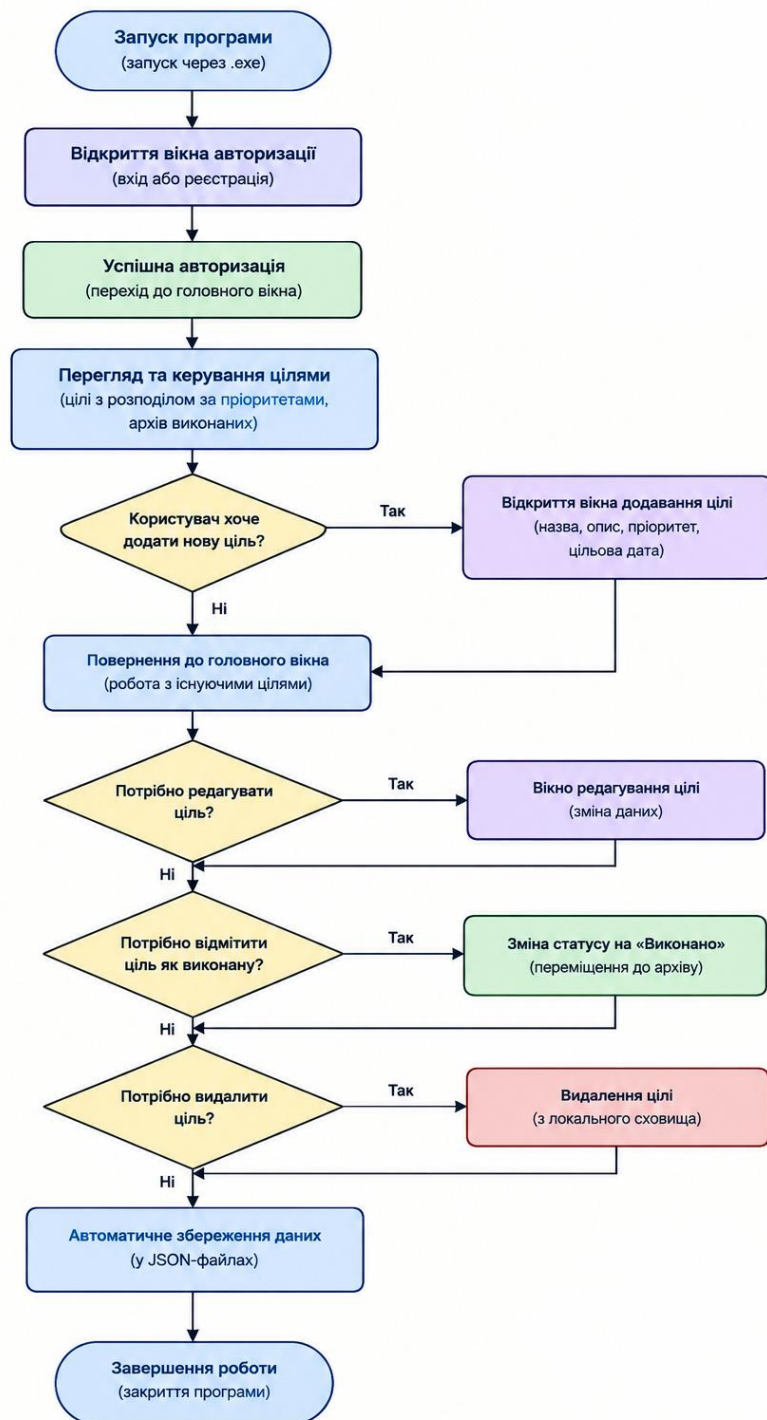
Загалом виконана робота дозволила не лише реалізувати поставлені завдання, але й отримати практичний досвід у розробці програмного забезпечення, роботі з графічними інтерфейсами, організації бізнес-логіки та структуризації програмних проєктів. Це сприяє формуванню професійних компетентностей, необхідних для подальшої роботи у сфері інформаційних технологій.

ПЕРЕЛІК ВИКОРИСТАНИХ ДЖЕРЕЛ

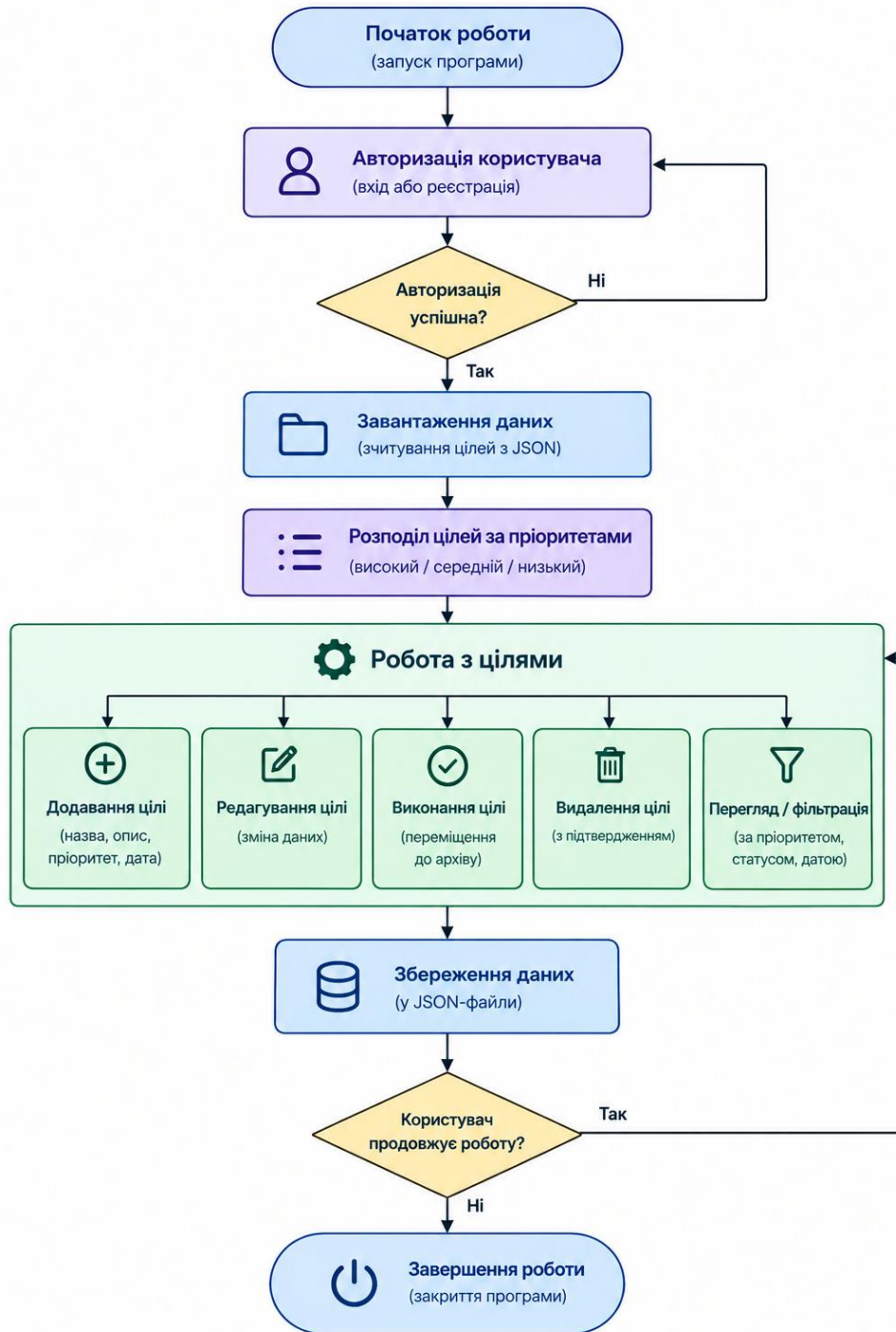
1. Introduction to WPF. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/overview/> (дата звернення: 11.04.2026).
2. XAML Overview. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/xaml/> (дата звернення: 11.04.2026).
3. WPF Controls Overview. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/wpf/controls/> (дата звернення: 12.04.2026).
4. C# documentation. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/> (дата звернення: 13.04.2026).
5. Object-oriented programming. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/tutorials/oop> (дата звернення: 14.04.2026).
6. ICommand Interface. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.windows.input.icommand> (дата звернення: 15.04.2026).
7. INotifyPropertyChanged Interface. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.componentmodel.inotifypropertychanged> (дата звернення: 16.04.2026).
8. System.Text.Json overview. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/standard/serialization/system-text-json/overview> (дата звернення: 17.04.2026).
9. File Class (System.IO). Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.io.file> (дата звернення: 18.04.2026).
10. Directory Class (System.IO). Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.io.directory> (дата звернення: 18.04.2026).
11. ObservableCollection Class. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.collections.objectmodel.observablecollection-1> (дата звернення: 19.04.2026).
12. DateTime Struct. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.datetime> (дата звернення: 20.04.2026).
13. LINQ (Language Integrated Query). Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/linq/> (дата звернення: 20.04.2026).
14. SHA256 Class. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/api/system.security.cryptography.sha256> (дата звернення: 21.04.2026).

15..NET 10 documentation. Microsoft Learn. URL: <https://learn.microsoft.com/en-us/dotnet/core/> (дата звернення: 22.04.2026).

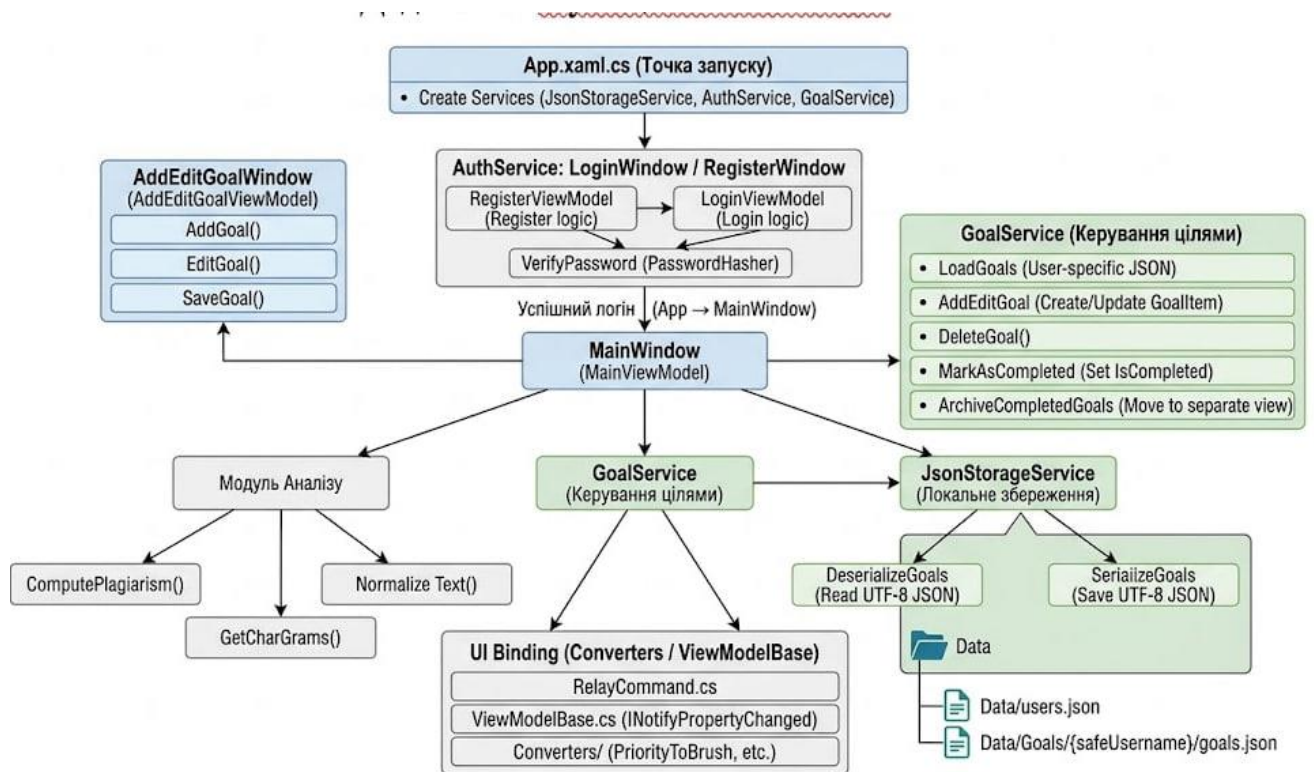
Додаток 1. Блок-схема сценарію роботи програми



Додаток 2. Блок-схеми алгоритму роботи з цілями



Додаток 3. Функціональна схема



Додаток 4. Лістинг програми

App.xaml

```
<Application x:Class="GoalTrackerApp.App"
<Application x:Class="Goal_Prioritizer.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:GoalTrackerApp">
    xmlns:local="clr-namespace:Goal_Prioritizer">
<Application.Resources>
    <ResourceDictionary>
        <ResourceDictionary.MergedDictionaries>
            <ResourceDictionary Source="Resources/DarkTheme.xaml"/>
        </ResourceDictionary.MergedDictionaries>
    </ResourceDictionary>
</Application.Resources>
</Application>
```

App.xaml.cs

```
using System.Windows;
using GoalTrackerApp.Services;
using GoalTrackerApp.ViewModels;
using GoalTrackerApp.Views;
using Goal_Prioritizer.Services;
using Goal_Prioritizer.ViewModels;
using Goal_Prioritizer.Views;

namespace GoalTrackerApp
namespace Goal_Prioritizer
{
    public partial class App : Application
    {
        private JsonStorageService? _storageService;
        private AuthService? _authService;
        private GoalService? _goalService;

        protected override void OnStartup(StartupEventArgs e)
        {
            base.OnStartup(e);

            _storageService = new JsonStorageService();
            _authService = new AuthService(_storageService);
            _goalService = new GoalService(_storageService);

            ShowLoginWindow();
        }
    }
}
```

```

    }

    public void ShowLoginWindow()
    {
        if (_authService == null || _goalService == null) return;

        var loginVm = new LoginViewModel(_authService);
        var loginWindow = new LoginWindow { DataContext = loginVm };

        loginVm.OnLoginSuccess += (user) =>
        {
            var mainVm = new MainViewModel(user, _goalService);
            var mainWindow = new MainWindow { DataContext = mainVm };

            mainVm.OnLogout += () =>
            {
                mainWindow.Close();
                ShowLoginWindow();
            };

            loginWindow.Close();
            mainWindow.Show();
        };

        loginVm.OnOpenRegister += () =>
        {
            var registerVm = new RegisterViewModel(_authService);
            var registerWindow = new RegisterWindow { DataContext = registerVm, Owner = loginWindow };

            registerVm.OnRegisterSuccess += () =>
            {
                registerWindow.Close();
            };

            registerVm.OnBackToLogin += () =>
            {
                registerWindow.Close();
            };

            registerWindow.ShowDialog();
        };
    };
};

```

```

        loginWindow.Show();
    }
}

```

RegisterWindow.xaml

```

<Window x:Class="GoalTrackerApp.Views.RegisterWindow"
<Window x:Class="Goal_Prioritizer.Views.RegisterWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Реєстрація акаунту | Система обліку цілей"
    Height="580" Width="440"
    WindowStartupLocation="CenterOwner"
    ResizeMode="NoResize"
    Style="{StaticResource DarkWindowStyle}">

    <Border Padding="35" Background="{StaticResource BgWindowBrush}">
        <StackPanel VerticalAlignment="Center">

            <!-- Title -->
            <TextBlock Text="Створення акаунту" Style="{StaticResource HeaderTextStyle}"
HorizontalAlignment="Center" Margin="0,0,0,5"/>
            <TextBlock Text="Заповніть дані для реєстрації" Style="{StaticResource SubtitleTextStyle}"
HorizontalAlignment="Center" Margin="0,0,0,20"/>

            <!-- Error Notification -->
            <Border Background="#3D1E28" BorderBrush="{StaticResource DangerBrush}" BorderThickness="1"
CornerRadius="8" Padding="10" Margin="0,0,0,15"
                Visibility="{Binding ErrorMessage, Converter={StaticResource
StringToVisConverter}}">
                <TextBlock Text="{Binding ErrorMessage}" Foreground="{StaticResource DangerBrush}"
TextWrapping="Wrap" FontSize="13" HorizontalAlignment="Center"/>
            </Border>

            <!-- Success Notification -->
            <Border Background="#1E3D28" BorderBrush="#A6E3A1" BorderThickness="1" CornerRadius="8"
Padding="10" Margin="0,0,0,15"
                Visibility="{Binding SuccessMessage, Converter={StaticResource
StringToVisConverter}}">
                <TextBlock Text="{Binding SuccessMessage}" Foreground="#A6E3A1" TextWrapping="Wrap"
FontSize="13" HorizontalAlignment="Center"/>
            </Border>

```

```

        <!-- Inputs -->
        <StackPanel Margin="0,0,0,15">
            <TextBlock Text="Лорін" Foreground="{StaticResource TextSecondaryBrush}" FontSize="13"
Margin="0,0,0,6"/>
            <TextBox Text="{Binding Username, UpdateSourceTrigger=PropertyChanged}"
                Style="{StaticResource DarkTextBoxStyle}"
                Margin="0,0,0,12"/>

            <TextBlock Text="Електронна пошта" Foreground="{StaticResource TextSecondaryBrush}"
FontSize="13" Margin="0,0,0,6"/>
            <TextBox Text="{Binding Email, UpdateSourceTrigger=PropertyChanged}"
                Style="{StaticResource DarkTextBoxStyle}"
                Margin="0,0,0,12"/>

            <TextBlock Text="Пароль" Foreground="{StaticResource TextSecondaryBrush}" FontSize="13"
Margin="0,0,0,6"/>
            <PasswordBox x:Name="TxtPassword"
                Style="{StaticResource DarkPasswordBoxStyle}"
                Margin="0,0,0,12"/>

            <TextBlock Text="Підтвердження пароля" Foreground="{StaticResource TextSecondaryBrush}"
FontSize="13" Margin="0,0,0,6"/>
            <PasswordBox x:Name="TxtConfirmPassword"
                Style="{StaticResource DarkPasswordBoxStyle}"/>
        </StackPanel>

        <!-- Buttons -->
        <Button Content="Зареєструватися"
            Style="{StaticResource PrimaryButtonStyle}"
            Height="44"
            Click="BtnRegister_Click"
            Margin="0,5,0,10"/>

        <Button Content="Скасувати / Повернутися"
            Style="{StaticResource SecondaryButtonStyle}"
            Height="40"
            Command="{Binding BackToLoginCommand}"/>

    </StackPanel>
</Border>
</Window>

```

```

RegisterWindow.xaml.cs
using System.Windows;
using GoalTrackerApp.ViewModels;
using Goal_Prioritizer.ViewModels;

namespace GoalTrackerApp.Views
namespace Goal_Prioritizer.Views
{
    public partial class RegisterWindow : Window
    {
        public RegisterWindow()
        {
            InitializeComponent();

            private void BtnRegister_Click(object sender, RoutedEventArgs e)
            {
                if (DataContext is RegisterViewModel vm)
                {
                    vm.ExecuteRegisterWithPasswords(TxtPassword.Password, TxtConfirmPassword.Password);
                }
            }
        }
    }
}

```

```

MainWindow.xaml
<Window x:Class="GoalTrackerApp.Views.MainWindow"
<Window x:Class="Goal_Prioritizer.Views.MainWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:models="clr-namespace:GoalTrackerApp.Models"
    xmlns:models="clr-namespace:Goal_Prioritizer.Models"
    Title="Система обліку особистих цілей із пріоритезацією"
    Height="760" Width="1100"
    WindowStartupLocation="CenterScreen"
    MinHeight="650" MinWidth="900"
    Style="{StaticResource DarkWindowStyle}">

    <Grid Margin="20">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>

```

```

        <RowDefinition Height="*" />
    </Grid.RowDefinitions>

    <!-- TOP BAR: User Info & Logout -->
    <Border Grid.Row="0" Background="{StaticResource BgCardBrush}" CornerRadius="12"
    Padding="20,14" Margin="0,0,0,15">
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*" />
                <ColumnDefinition Width="Auto" />
            </Grid.ColumnDefinitions>

            <StackPanel Grid.Column="0" Orientation="Horizontal" VerticalAlignment="Center">
                <Border Width="40" Height="40" Background="{StaticResource AccentPrimaryBrush}"
                CornerRadius="10" Margin="0,0,14,0">
                    <TextBlock Text="□" FontSize="20" HorizontalAlignment="Center"
                    VerticalAlignment="Center" />
                </Border>
                <StackPanel VerticalAlignment="Center">
                    <TextBlock Text="{Binding WelcomeMessage}" FontSize="18" FontWeight="Bold"
                    Foreground="{StaticResource TextPrimaryBrush}" />
                    <TextBlock Text="Система обліку особистих цілей" FontSize="13"
                    Foreground="{StaticResource TextSecondaryBrush}" />
                </StackPanel>
            </StackPanel>

            <Button Grid.Column="1"
            Content="□ Вийти"
            Style="{StaticResource SecondaryButtonStyle}"
            Command="{Binding LogoutCommand}"
            VerticalAlignment="Center" />
        </Grid>
    </Border>

    <!-- SEARCH BAR -->
    <Grid Grid.Row="1" Margin="0,0,0,15">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*" />
        </Grid.ColumnDefinitions>

        <TextBox Grid.Column="0"
        Text="{Binding SearchQuery, UpdateSourceTrigger=PropertyChanged}"
        Style="{StaticResource DarkTextBoxStyle}"
        Tag="□ Пошук за назвою або описом..." />
    </Grid>

```

```

<!-- MAIN CONTENT TABS -->
<TabControl Grid.Row="2" Style="{StaticResource DarkTabControlStyle}">

    <!-- TAB 1: ACTIVE GOALS - 3 PRIORITY COLUMNS -->
    <TabItem Header="□ Активні цілі" Style="{StaticResource DarkTabItemStyle}">
        <Grid Margin="0,15,0,0">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*/>
                <ColumnDefinition Width="15"/>
                <ColumnDefinition Width="*/>
                <ColumnDefinition Width="15"/>
                <ColumnDefinition Width="*/>
            </Grid.ColumnDefinitions>

            <!-- COLUMN 1: HIGH PRIORITY (Високий) -->
            <Border Grid.Column="0" Background="{StaticResource BgCardBrush}" CornerRadius="12"
                Padding="14" BorderBrush="{StaticResource BorderDarkBrush}" BorderThickness="1">
                <Grid>
                    <Grid.RowDefinitions>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="*/>
                        <RowDefinition Height="Auto"/>
                    </Grid.RowDefinitions>

                    <!-- Header Column 1 -->
                    <Border Grid.Row="0" Background="#3D1E28" BorderBrush="#F38BA8"
                        BorderThickness="1" CornerRadius="8" Padding="12,10" Margin="0,0,0,12">
                        <Grid>
                            <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
                                <TextBlock Text="□" FontSize="15" Margin="0,0,8,0"/>
                                <TextBlock Text="Високий пріоритет" FontSize="15"
                                    FontWeight="Bold" Foreground="#F38BA8"/>
                            </StackPanel>
                            <Border HorizontalAlignment="Right" Background="#F38BA8"
                                CornerRadius="10" Padding="8,2">
                                <TextBlock Text="{Binding HighPriorityGoals.Count}"
                                    FontSize="12" FontWeight="Bold" Foreground="#11111B"/>
                            </Border>
                        </Grid>
                    </Border>

                    <!-- List Column 1 -->
                    <ScrollViewer Grid.Row="1" VerticalScrollBarVisibility="Auto"
                        Margin="0,0,0,10">
                        <ItemsControl ItemsSource="{Binding HighPriorityGoals}">
                            <ItemsControl.ItemTemplate>
                                <DataTemplate>

```

```

        <Border Background="#181825" CornerRadius="8" Padding="12"
Margin="0,0,0,10" BorderBrush="#F38BA8" BorderThickness="1">
            <Grid>
                <Grid.RowDefinitions>
                    <RowDefinition Height="Auto"/>
                    <RowDefinition Height="Auto"/>
                    <RowDefinition Height="Auto"/>
                </Grid.RowDefinitions>

                <Grid Grid.Row="0" Margin="0,0,0,6">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="Auto"/>
                        <ColumnDefinition Width="*/>
                        <ColumnDefinition Width="Auto"/>
                    </Grid.ColumnDefinitions>

                    <CheckBox Grid.Column="0" IsChecked="{Binding
IsCompleted, Mode=TwoWay}" Style="{StaticResource DarkCheckBoxStyle}" VerticalAlignment="Center"
Margin="0,0,8,0"/>

                    <TextBlock Grid.Column="1" Text="{Binding
Title}" FontSize="15" FontWeight="Bold" Foreground="{StaticResource TextPrimaryBrush}"
TextWrapping="Wrap" VerticalAlignment="Center"/>

                    <StackPanel Grid.Column="2"
Orientation="Horizontal">
                        <Button Content="□" Style="{StaticResource
SecondaryButtonStyle}" Padding="5,3" Margin="0,0,4,0" Command="{Binding DataContext.EditGoalCommand,
RelativeSource={RelativeSource AncestorType=Window}}" CommandParameter="{Binding}"/>
                        <Button Content="□" Style="{StaticResource
DangerButtonStyle}" Padding="5,3" Command="{Binding DataContext.DeleteGoalCommand,
RelativeSource={RelativeSource AncestorType=Window}}" CommandParameter="{Binding}"/>
                    </StackPanel>
                </Grid>

                <TextBlock Grid.Row="1" Text="{Binding
Description}" FontSize="13" Foreground="{StaticResource TextSecondaryBrush}" TextWrapping="Wrap"
Margin="30,0,0,6" Visibility="{Binding Description, Converter={StaticResource StringToVisConverter}}"/>

                <StackPanel Grid.Row="2" Orientation="Horizontal"
Margin="30,2,0,0">
                    <TextBlock Text="□ " FontSize="11"
Foreground="{StaticResource TextMutedBrush}"/>
                    <TextBlock Text="{Binding TargetDate,
Converter={StaticResource DateFormatConverter}}" FontSize="11" Foreground="{StaticResource
TextSecondaryBrush}"/>
                </StackPanel>
            </Grid>

```

```

        </Grid>
    </Border>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</ScrollView>

<!-- Add Button Column 1 -->
<Button Grid.Row="2"
        Content="+ Додати задачу"
        Style="{StaticResource PrimaryButtonStyle}"
        Height="38"
        Command="{Binding AddGoalWithPriorityCommand}"
        CommandParameter="{x:Static models:PriorityLevel.High}"/>
</Grid>
</Border>

<!-- COLUMN 2: MEDIUM PRIORITY (Середній) -->
<Border Grid.Column="2" Background="{StaticResource BgCardBrush}" CornerRadius="12"
        Padding="14" BorderBrush="{StaticResource BorderDarkBrush}" BorderThickness="1">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!-- Header Column 2 -->
        <Border Grid.Row="0" Background="#3D2E1E" BorderBrush="#FAB387"
            BorderThickness="1" CornerRadius="8" Padding="12,10" Margin="0,0,0,12">
            <Grid>
                <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
                    <TextBlock Text="⚡" FontSize="15" Margin="0,0,8,0"/>
                    <TextBlock Text="Середній пріоритет" FontSize="15"
                        FontWeight="Bold" Foreground="#FAB387"/>
                </StackPanel>
                <Border HorizontalAlignment="Right" Background="#FAB387"
                    CornerRadius="10" Padding="8,2">
                    <TextBlock Text="{Binding MediumPriorityGoals.Count}"
                        FontSize="12" FontWeight="Bold" Foreground="#11111B"/>
                </Border>
            </Grid>
        </Border>

        <!-- List Column 2 -->
        <ScrollView Grid.Row="1" VerticalScrollBarVisibility="Auto"
            Margin="0,0,0,10">

```

```

<ItemsControl ItemsSource="{Binding MediumPriorityGoals}">
    <ItemsControl.ItemTemplate>
        <DataTemplate>
            <Border Background="#181825" CornerRadius="8" Padding="12"
Margin="0,0,0,10" BorderBrush="#FAB387" BorderThickness="1">
                <Grid>
                    <Grid.RowDefinitions>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="Auto"/>
                        <RowDefinition Height="Auto"/>
                    </Grid.RowDefinitions>

                    <Grid Grid.Row="0" Margin="0,0,0,6">
                        <Grid.ColumnDefinitions>
                            <ColumnDefinition Width="Auto"/>
                            <ColumnDefinition Width="*/>
                            <ColumnDefinition Width="Auto"/>
                        </Grid.ColumnDefinitions>

                        <CheckBox Grid.Column="0" IsChecked="{Binding
IsCompleted, Mode=TwoWay}" Style="{StaticResource DarkCheckBoxStyle}" VerticalAlignment="Center"
Margin="0,0,8,0"/>

                        <TextBlock Grid.Column="1" Text="{Binding
Title}" FontSize="15" FontWeight="Bold" Foreground="{StaticResource TextPrimaryBrush}"
TextWrapping="Wrap" VerticalAlignment="Center"/>

                        <StackPanel Grid.Column="2"
Orientation="Horizontal">
                            <Button Content="□" Style="{StaticResource
SecondaryButtonStyle}" Padding="5,3" Margin="0,0,4,0" Command="{Binding DataContext.EditGoalCommand,
RelativeSource={RelativeSource AncestorType=Window}}" CommandParameter="{Binding}"/>
                            <Button Content="□" Style="{StaticResource
DangerButtonStyle}" Padding="5,3" Command="{Binding DataContext.DeleteGoalCommand,
RelativeSource={RelativeSource AncestorType=Window}}" CommandParameter="{Binding}"/>
                        </StackPanel>
                    </Grid>

                    <TextBlock Grid.Row="1" Text="{Binding
Description}" FontSize="13" Foreground="{StaticResource TextSecondaryBrush}" TextWrapping="Wrap"
Margin="30,0,0,6" Visibility="{Binding Description, Converter={StaticResource StringToVisConverter}}"/>

                    <StackPanel Grid.Row="2" Orientation="Horizontal"
Margin="30,2,0,0">
                        <TextBlock Text="□" FontSize="11"
Foreground="{StaticResource TextMutedBrush}"/>

```

```

        <TextBlock Text="{Binding TargetDate,
Converter={StaticResource DateFormatConverter}}" FontSize="11" Foreground="{StaticResource
TextSecondaryBrush}"/>

        </StackPanel>

    </Grid>

</Border>

</DataTemplate>

</ItemsControl.ItemTemplate>

</ItemsControl>

</ScrollView>

<!-- Add Button Column 2 -->
<Button Grid.Row="2"
        Content="+ Додати задачу"
        Style="{StaticResource PrimaryButtonStyle}"
        Height="38"
        Command="{Binding AddGoalWithPriorityCommand}"
        CommandParameter="{x:Static models:PriorityLevel.Medium}"/>

</Grid>
</Border>

<!-- COLUMN 3: LOW PRIORITY (Низький - Жовтий) -->
<Border Grid.Column="4" Background="{StaticResource BgCardBrush}" CornerRadius="12"
Padding="14" BorderBrush="{StaticResource BorderDarkBrush}" BorderThickness="1">
    <Grid>
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>

        <!-- Header Column 3 -->
        <Border Grid.Row="0" Background="#3D391E" BorderBrush="#F9E2AF"
BorderThickness="1" CornerRadius="8" Padding="12,10" Margin="0,0,0,12">
            <Grid>
                <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
                    <TextBlock Text="□" FontSize="15" Margin="0,0,8,0"/>
                    <TextBlock Text="Низький пріоритет" FontSize="15"
FontWeight="Bold" Foreground="#F9E2AF"/>
                </StackPanel>
                <Border HorizontalAlignment="Right" Background="#F9E2AF"
CornerRadius="10" Padding="8,2">
                    <TextBlock Text="{Binding LowPriorityGoals.Count}"
FontSize="12" FontWeight="Bold" Foreground="#11111B"/>
                </Border>
            </Grid>
        </Border>
    </Grid>
</Border>

```

```

<!-- List Column 3 -->
<ScrollView Grid.Row="1" VerticalScrollBarVisibility="Auto"
Margin="0,0,0,10">
    <ItemsControl ItemsSource="{Binding LowPriorityGoals}">
        <ItemsControl.ItemTemplate>
            <DataTemplate>
                <Border Background="#181825" CornerRadius="8" Padding="12"
Margin="0,0,0,10" BorderBrush="#F9E2AF" BorderThickness="1">
                    <Grid>
                        <Grid.RowDefinitions>
                            <RowDefinition Height="Auto"/>
                            <RowDefinition Height="Auto"/>
                            <RowDefinition Height="Auto"/>
                        </Grid.RowDefinitions>

                        <Grid Grid.Row="0" Margin="0,0,0,6">
                            <Grid.ColumnDefinitions>
                                <ColumnDefinition Width="Auto"/>
                                <ColumnDefinition Width="*/>
                                <ColumnDefinition Width="Auto"/>
                            </Grid.ColumnDefinitions>

                                <CheckBox Grid.Column="0" IsChecked="{Binding
IsCompleted, Mode=TwoWay}" Style="{StaticResource DarkCheckBoxStyle}" VerticalAlignment="Center"
Margin="0,0,8,0"/>

                                <TextBlock Grid.Column="1" Text="{Binding
Title}" FontSize="15" FontWeight="Bold" Foreground="{StaticResource TextPrimaryBrush}"
TextWrapping="Wrap" VerticalAlignment="Center"/>

                                <StackPanel Grid.Column="2"
Orientation="Horizontal">
                                    <Button Content="□" Style="{StaticResource
SecondaryButtonStyle}" Padding="5,3" Margin="0,0,4,0" Command="{Binding DataContext.EditGoalCommand,
RelativeSource={RelativeSource AncestorType=Window}}" CommandParameter="{Binding}"/>
                                    <Button Content="□" Style="{StaticResource
DangerButtonStyle}" Padding="5,3" Command="{Binding DataContext.DeleteGoalCommand,
RelativeSource={RelativeSource AncestorType=Window}}" CommandParameter="{Binding}"/>
                                </StackPanel>
                            </Grid>

                        <TextBlock Grid.Row="1" Text="{Binding
Description}" FontSize="13" Foreground="{StaticResource TextSecondaryBrush}" TextWrapping="Wrap"
Margin="30,0,0,6" Visibility="{Binding Description, Converter={StaticResource StringToVisConverter}}"/>

```

```

<StackPanel Grid.Row="2" Orientation="Horizontal"
Margin="30,2,0,0">
    <TextBlock Text="□ " FontSize="11"
Foreground="{StaticResource TextMutedBrush}"/>
    <TextBlock Text="{Binding TargetDate,
Converter={StaticResource DateFormatConverter}}" FontSize="11" Foreground="{StaticResource
TextSecondaryBrush}"/>
    </StackPanel>
</Grid>
</Border>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</ScrollView>

<!-- Add Button Column 3 -->
<Button Grid.Row="2"
Content="+ Додати задачу"
Style="{StaticResource PrimaryButtonStyle}"
Height="38"
Command="{Binding AddGoalWithPriorityCommand}"
CommandParameter="{x:Static models:PriorityLevel.Low}"/>
</Grid>
</Border>

</Grid>
</TabItem>

<!-- TAB 2: ARCHIVE (COMPLETED GOALS) -->
<TabItem Header="□ Архів виконаних задач" Style="{StaticResource DarkTabItemStyle}">
    <Grid Margin="0,15,0,0">
        <ScrollView VerticalScrollBarVisibility="Auto">
            <ItemsControl ItemsSource="{Binding CompletedGoals}">
                <ItemsControl.ItemTemplate>
                    <DataTemplate>
                        <Border Background="#161622" CornerRadius="10" Padding="16,12"
Margin="0,0,0,10" BorderBrush="{StaticResource BorderDarkBrush}" BorderThickness="1" Opacity="0.85">
                            <Grid>
                                <Grid.ColumnDefinitions>
                                    <ColumnDefinition Width="Auto"/>
                                    <ColumnDefinition Width="*/>
                                    <ColumnDefinition Width="Auto"/>
                                    <ColumnDefinition Width="Auto"/>
                                </Grid.ColumnDefinitions>

                                <!-- Checkbox (Uncheck to restore) -->

```

```

<CheckBox Grid.Column="0" IsChecked="{Binding IsCompleted,
Mode=TwoWay}" Style="{StaticResource DarkCheckBoxStyle}" VerticalAlignment="Center" Margin="0,0,15,0"
ToolTip="Зняти позначку (повернути до активних)"/>

<!-- Title and Description -->
<StackPanel Grid.Column="1" VerticalAlignment="Center"
Margin="0,0,15,0">
    <TextBlock Text="{Binding Title}" FontSize="16"
    FontWeight="SemiBold" Foreground="{StaticResource TextSecondaryBrush}" TextDecorations="Strikethrough"
    TextWrapping="Wrap"/>
    <TextBlock Text="{Binding Description}" FontSize="13"
    Foreground="{StaticResource TextMutedBrush}" TextWrapping="Wrap" Margin="0,4,0,0" Visibility="{Binding
    Description, Converter={StaticResource StringToVisConverter}}"/>

    <StackPanel Orientation="Horizontal" Margin="0,6,0,0">
        <TextBlock Text="✓ Виконано: " FontSize="12"
        Foreground="{StaticResource TextMutedBrush}"/>
        <TextBlock Text="{Binding CompletedAt,
        Converter={StaticResource DateFormatConverter}}" FontSize="12" Foreground="#A6E3A1"/>
    </StackPanel>
</StackPanel>

<!-- Priority Badge -->
<Border Grid.Column="2" Style="{StaticResource
PriorityBadgeStyle}" Margin="0,0,15,0">
    <TextBlock Text="{Binding PriorityName}"
    Style="{StaticResource PriorityTextStyle}"/>
</Border>

<!-- Delete Action Button -->
<StackPanel Grid.Column="3" Orientation="Horizontal"
VerticalAlignment="Center">
    <Button Content="✖" Style="{StaticResource
DangerButtonStyle}" Padding="8,6" ToolTip="Видалити з архіву" Command="{Binding
DataContext.DeleteGoalCommand, RelativeSource={RelativeSource AncestorType=Window}}"
CommandParameter="{Binding}"/>
</StackPanel>
</Grid>
</Border>
</DataTemplate>
</ItemsControl.ItemTemplate>
</ItemsControl>
</ScrollViewer>
</Grid>
</TabItem>

```

```

        </TabControl>

    </Grid>
</Window>

MainWindow.xaml.cs
using System.Windows;
using GoalTrackerApp.ViewModels;
using Goal_Prioritizer.ViewModels;

namespace GoalTrackerApp.Views
namespace Goal_Prioritizer.Views
{
    public partial class MainWindow : Window
    {
        public MainWindow()
        {
            InitializeComponent();
        }

        protected override void OnPropertyChanged(DependencyPropertyChangedEventArgs e)
        {
            base.OnPropertyChanged(e);
            if (e.Property == DataContextProperty && e.NewValue is MainViewModel vm)
            {
                vm.OnOpenAddGoalDialog += ShowAddGoalDialog;
                vm.OnOpenEditGoalDialog += ShowEditGoalDialog;
            }
        }

        private void ShowAddGoalDialog(AddEditGoalViewModel addVm)
        {
            var dialog = new AddEditGoalWindow { DataContext = addVm, Owner = this };
            dialog.ShowDialog();
        }

        private void ShowEditGoalDialog(AddEditGoalViewModel editVm)
        {
            var dialog = new AddEditGoalWindow { DataContext = editVm, Owner = this };
            dialog.ShowDialog();
        }
    }
}

```

LoginWindow.xaml

```
<Window x:Class="GoalTrackerApp.Views.LoginWindow"
<Window x:Class="Goal_Prioritizer.Views.LoginWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="Авторизація | Система обліку цілей"
    Height="520" Width="440"
    WindowStartupLocation="CenterScreen"
    ResizeMode="NoResize"
    Style="{StaticResource DarkWindowStyle}">

    <Border Padding="35" Background="{StaticResource BgWindowBrush}">
        <StackPanel VerticalAlignment="Center">

            <!-- App Logo / Title -->
            <StackPanel HorizontalAlignment="Center" Margin="0,0,0,25">
                <Border Width="60" Height="60" Background="{StaticResource AccentPrimaryBrush}"
                CornerRadius="16" Margin="0,0,0,12">
                    <TextBlock Text="□" FontSize="30" HorizontalAlignment="Center"
                    VerticalAlignment="Center"/>
                </Border>
                <TextBlock Text="Вхід у систему" Style="{StaticResource HeaderTextStyle}"
                HorizontalAlignment="Center" Margin="0"/>
                <TextBlock Text="Облік особистих цілей із пріоритезацією" Style="{StaticResource
                SubtitleTextStyle}" HorizontalAlignment="Center" Margin="0,4,0,0"/>
            </StackPanel>

            <!-- Error Notification -->
            <Border Background="#3D1E28" BorderBrush="{StaticResource DangerBrush}" BorderThickness="1"
            CornerRadius="8" Padding="10" Margin="0,0,0,15"
                Visibility="{Binding ErrorMessage, Converter={StaticResource
                StringToVisConverter}}">
                <TextBlock Text="{Binding ErrorMessage}" Foreground="{StaticResource DangerBrush}"
                TextWrapping="Wrap" FontSize="13" HorizontalAlignment="Center"/>
            </Border>

            <!-- Input Fields -->
            <StackPanel Margin="0,0,0,20">
                <TextBlock Text="Логін або Email" Foreground="{StaticResource TextSecondaryBrush}"
                FontSize="13" Margin="0,0,0,6"/>
                <TextBox Text="{Binding UsernameOrEmail, UpdateSourceTrigger=PropertyChanged}"
                Style="{StaticResource DarkTextBoxStyle}"
                Margin="0,0,0,16"/>
            </StackPanel>
        </StackPanel>
    </Border>
</Window>
```

```

        <TextBlock Text="Пароль" Foreground="{StaticResource TextSecondaryBrush}" FontSize="13"
Margin="0,0,0,6"/>
        <PasswordBox x:Name="TxtPassword"
            Style="{StaticResource DarkPasswordBoxStyle}"
            PasswordChanged="TxtPassword_PasswordChanged"/>
    </StackPanel>

    <!-- Login Button -->
    <Button Content="Увійти"
        Style="{StaticResource PrimaryButtonStyle}"
        Height="44"
        Click="BtnLogin_Click"
        Margin="0,0,0,20"/>

    <!-- Register Link -->
    <StackPanel Orientation="Horizontal" HorizontalAlignment="Center">
        <StackPanel Orientation="Horizontal" HorizontalAlignment="Center" Visibility="{Binding
ShowRegisterLink, Converter={StaticResource BoolToVisConverter}}">
            <TextBlock Text="Не маєте акаунту? " Foreground="{StaticResource TextMutedBrush}"
FontSize="13"/>
            <TextBlock Text="Створити акаунт"
                Foreground="{StaticResource AccentPrimaryBrush}"
                FontSize="13"
                FontWeight="SemiBold"
                Cursor="Hand">
                <TextBlock.InputBindings>
                    <MouseBinding MouseAction="LeftClick" Command="{Binding OpenRegisterCommand}"/>
                </TextBlock.InputBindings>
                <TextBlock.Style>
                    <Style TargetType="TextBlock">
                        <Style.Triggers>
                            <Trigger Property="IsMouseOver" Value="True">
                                <Setter Property="TextDecorations" Value="Underline"/>
                            </Trigger>
                        </Style.Triggers>
                    </Style>
                </TextBlock.Style>
            </TextBlock>
        </StackPanel>
    </StackPanel>
</Border>
</Window>

```

```

LoginWindow.xaml.cs
using System.Windows;
using GoalTrackerApp.ViewModels;
using Goal_Prioritizer.ViewModels;

namespace GoalTrackerApp.Views
namespace Goal_Prioritizer.Views
{
    public partial class LoginWindow : Window
    {
        public LoginWindow()
        {
            InitializeComponent();
        }

        private void BtnLogin_Click(object sender, RoutedEventArgs e)
        {
            if (DataContext is LoginViewModel vm)
            {
                vm.ExecuteLoginWithPassword(TxtPassword.Password);
            }
        }

        private void TxtPassword_PasswordChanged(object sender, RoutedEventArgs e)
        {
            // У разі необхідності реактивної валідації
        }
    }
}

```

```

AddEditGoalWindow.xaml
<Window x:Class="GoalTrackerApp.Views.AddEditGoalWindow"
<Window x:Class="Goal_Prioritizer.Views.AddEditGoalWindow"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    Title="{Binding WindowTitle}"
    Height="500" Width="460"
    WindowStartupLocation="CenterOwner"
    ResizeMode="NoResize"
    Style="{StaticResource DarkWindowStyle}">

    <Border Padding="30" Background="{StaticResource BgWindowBrush}">
        <Grid>

```

```

<Grid.RowDefinitions>
    <RowDefinition Height="Auto"/>
    <RowDefinition Height="*/>
    <RowDefinition Height="Auto"/>
</Grid.RowDefinitions>

<!-- Header -->
<TextBlock Grid.Row="0" Text="{Binding WindowTitle}" Style="{StaticResource
HeaderTextStyle}"/>

<!-- Content Form -->
<StackPanel Grid.Row="1">

    <!-- Error Notification -->
    <Border Background="#3D1E28" BorderBrush="{StaticResource DangerBrush}"
BorderThickness="1" CornerRadius="8" Padding="10" Margin="0,0,0,15"
    Visibility="{Binding ErrorMessage, Converter={StaticResource
StringToVisConverter}}">
        <TextBlock Text="{Binding ErrorMessage}" Foreground="{StaticResource DangerBrush}"
TextWrapping="Wrap" FontSize="13" HorizontalAlignment="Center"/>
    </Border>

    <!-- Title Input -->
    <TextBlock Text="Назва ціли *" Foreground="{StaticResource TextSecondaryBrush}"
FontSize="13" Margin="0,0,0,6"/>
    <TextBox Text="{Binding Title, UpdateSourceTrigger=PropertyChanged}"
        Style="{StaticResource DarkTextBoxStyle}"
        Margin="0,0,0,14"/>

    <!-- Description Input -->
    <TextBlock Text="Опис (необов'язково)" Foreground="{StaticResource TextSecondaryBrush}"
FontSize="13" Margin="0,0,0,6"/>
    <TextBox Text="{Binding Description, UpdateSourceTrigger=PropertyChanged}"
        Style="{StaticResource DarkTextBoxStyle}"
        AcceptsReturn="True"
        TextWrapping="Wrap"
        Height="70"
        VerticalContentAlignment="Top"
        Margin="0,0,0,14"/>

    <!-- Priority Selector -->
    <Grid Margin="0,0,0,14">
        <Grid.ColumnDefinitions>
            <ColumnDefinition Width="*/>
            <ColumnDefinition Width="*/>
        </Grid.ColumnDefinitions>

```

```

        <StackPanel Grid.Column="0" Margin="0,0,8,0">
            <TextBlock Text="Пріоритет" Foreground="{StaticResource TextSecondaryBrush}"
FontSize="13" Margin="0,0,0,6"/>
            <ComboBox ItemsSource="{Binding Priorities}"
                SelectedItem="{Binding SelectedPriority}"
                Style="{StaticResource DarkComboBoxStyle}">
                <ComboBox.ItemTemplate>
                    <DataTemplate>
                        <TextBlock Text="{Binding Converter={StaticResource
PriorityTextConverter}}"
                                Foreground="{Binding Converter={StaticResource
PriorityFgConverter}}"
                                FontWeight="SemiBold"/>
                    </DataTemplate>
                </ComboBox.ItemTemplate>
            </ComboBox>
        </StackPanel>

        <StackPanel Grid.Column="1" Margin="8,0,0,0">
            <TextBlock Text="Кінцева дата" Foreground="{StaticResource TextSecondaryBrush}"
FontSize="13" Margin="0,0,0,6"/>
            <DatePicker SelectedDate="{Binding TargetDate}"
                Style="{StaticResource DarkDatePickerStyle}"/>
        </StackPanel>
    </Grid>

</StackPanel>

<!-- Buttons -->
<Grid Grid.Row="2" Margin="0,15,0,0">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="12" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>

    <Button Grid.Column="0"
        Content="Скасувати"
        Style="{StaticResource SecondaryButtonStyle}"
        Height="42"
        Command="{Binding CancelCommand}"/>

    <Button Grid.Column="2"
        Content="Зберегти"
        Style="{StaticResource PrimaryButtonStyle}"

```

```

                Height="42"
                Command="{Binding SaveCommand}"/>
            </Grid>
        </Grid>
    </Border>
</Window>

```

AddEditGoalWindow.xaml.cs

```

using System.Windows;
using GoalTrackerApp.ViewModels;
using Goal_Prioritizer.ViewModels;

```

```

namespace GoalTrackerApp.Views
namespace Goal_Prioritizer.Views

```

```

{
    public partial class AddEditGoalWindow : Window
    {
        public AddEditGoalWindow()
        {
            InitializeComponent();
        }

        protected override void OnPropertyChanged(DependencyPropertyChangedEventArgs e)
        {
            base.OnPropertyChanged(e);
            if (e.Property == DataContextProperty && e.NewValue is AddEditGoalViewModel vm)
            {
                vm.OnSave += (_) => DialogResult = true;
                vm.OnCancel += () => DialogResult = false;
            }
        }
    }
}

```

ViewModelBase.cs

```

using System.ComponentModel;
using System.Runtime.CompilerServices;

```

```

namespace GoalTrackerApp.ViewModels
namespace Goal_Prioritizer.ViewModels
{

```

```

public abstract class ViewModelBase : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected virtual void OnPropertyChanged([CallerMemberName] string? propertyName = null)
    {
        PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(propertyName));
    }

    protected bool SetField<T>(ref T field, T value, [CallerMemberName] string? propertyName =
null)
    {
        if (EqualityComparer<T>.Default.Equals(field, value)) return false;
        field = value;
        OnPropertyChanged(propertyName);
        return true;
    }
}

```

RelayCommand.cs

```
using System;
```

```
using System.Windows.Input;
```

```
namespace GoalTrackerApp.ViewModels
```

```
namespace Goal_Prioritizer.ViewModels
```

```

{
    public class RelayCommand : ICommand
    {
        private readonly Action<object?> _execute;
        private readonly Predicate<object??> _canExecute;

        public RelayCommand(Action<object?> execute, Predicate<object??> canExecute = null)
        {
            _execute = execute ?? throw new ArgumentNullException(nameof(execute));
            _canExecute = canExecute;
        }

        public RelayCommand(Action execute, Func<bool>? canExecute = null)
            : this(_ => execute(), canExecute == null ? null : _ => canExecute())
        {
        }
    }
}

```

```

    public bool CanExecute(object? parameter)
    {
        return _canExecute == null || _canExecute(parameter);
    }

    public void Execute(object? parameter)
    {
        _execute(parameter);
    }

    public event EventHandler? CanExecuteChanged
    {
        add => CommandManager.RequerySuggested += value;
        remove => CommandManager.RequerySuggested -= value;
    }

    public void RaiseCanExecuteChanged()
    {
        CommandManager.InvalidateRequerySuggested();
    }
}
}

```

RegisterViewModel.cs

```

using System;
using System.Windows.Input;
using GoalTrackerApp.Services;
using Goal_Prioritizer.Services;

namespace GoalTrackerApp.ViewModels
namespace Goal_Prioritizer.ViewModels
{
    // No-op update to ensure file is updated.
    public class RegisterViewModel : ViewModelBase
    {
        private readonly AuthService _authService;
        private string _username = string.Empty;
        private string _email = string.Empty;
        private string _errorMessage = string.Empty;
        private string _successMessage = string.Empty;

        public event Action? OnRegisterSuccess;
        public event Action? OnBackToLogin;
    }
}

```

```

public RegisterViewModel(AuthService authService)
{
    _authService = authService;
    RegisterCommand = new RelayCommand(ExecuteRegister);
    BackToLoginCommand = new RelayCommand(ExecuteBackToLogin);
}

public string Username
{
    get => _username;
    set => SetField(ref _username, value);
}

public string Email
{
    get => _email;
    set => SetField(ref _email, value);
}

public string ErrorMessage
{
    get => _errorMessage;
    set => SetField(ref _errorMessage, value);
}

public string SuccessMessage
{
    get => _successMessage;
    set => SetField(ref _successMessage, value);
}

public ICommand RegisterCommand { get; }
public ICommand BackToLoginCommand { get; }

public void ExecuteRegisterWithPasswords(string password, string confirmPassword)
{
    ErrorMessage = string.Empty;
    SuccessMessage = string.Empty;

    if (password != confirmPassword)
    {
        ErrorMessage = "Паролі не збігаються.";
        return;
    }
}

```

```

    }

    if (_authService.Register(Username, Email, password, out string error))
    {
        SuccessMessage = "Акаунт успішно створено! Повертаємось до входу...";
        OnRegisterSuccess?.Invoke();
    }
    else
    {
        ErrorMessage = error;
    }
}

private void ExecuteRegister(object? parameter)
{
    // Параметри можуть передаватися з в'юшки
}

private void ExecuteBackToLogin()
{
    OnBackToLogin?.Invoke();
}
}
}

```

```

MainViewModel.cs
using System;
using System.Collections.ObjectModel;
using System.Linq;
using System.Windows.Input;
using GoalTrackerApp.Models;
using GoalTrackerApp.Services;
using Goal_Prioritizer.Models;
using Goal_Prioritizer.Services;

namespace GoalTrackerApp.ViewModels
namespace Goal_Prioritizer.ViewModels
{
    public class MainViewModel : ViewModelBase
    {
        private readonly GoalService _goalService;
        private readonly User _currentUser;
        private string _searchQuery = string.Empty;
    }
}

```

```

public ObservableCollection<GoalItemViewModel> HighPriorityGoals { get; } = new();
public ObservableCollection<GoalItemViewModel> MediumPriorityGoals { get; } = new();
public ObservableCollection<GoalItemViewModel> LowPriorityGoals { get; } = new();
public ObservableCollection<GoalItemViewModel> CompletedGoals { get; } = new();

public event Action? OnLogout;
public event Action<AddEditGoalViewModel>? OnOpenAddGoalDialog;
public event Action<AddEditGoalViewModel>? OnOpenEditGoalDialog;

public MainViewModel(User currentUser, GoalService goalService)
{
    _currentUser = currentUser;
    _goalService = goalService;

    AddGoalCommand = new RelayCommand(ExecuteAddGoal);
    AddGoalWithPriorityCommand = new RelayCommand(ExecuteAddGoalWithPriority);
    EditGoalCommand = new RelayCommand(ExecuteEditGoal);
    DeleteGoalCommand = new RelayCommand(ExecuteDeleteGoal);
    LogoutCommand = new RelayCommand(ExecuteLogout);

    LoadGoals();
}

public User CurrentUser => _currentUser;
public string WelcomeMessage => $"Битраємо, {_currentUser.Username}!";

public string SearchQuery
{
    get => _searchQuery;
    set
    {
        if (SetField(ref _searchQuery, value))
        {
            ApplyFilters();
        }
    }
}

public ICommand AddGoalCommand { get; }
public ICommand AddGoalWithPriorityCommand { get; }
public ICommand EditGoalCommand { get; }
public ICommand DeleteGoalCommand { get; }
public ICommand LogoutCommand { get; }

```

```

public void LoadGoals()
{
    ApplyFilters();
}

private void ApplyFilters()
{
    var goals = _goalService.GetGoalsForUser(_currentUser.Username);

    // Фільтрація по пошуковому запиту
    if (!string.IsNullOrEmpty(SearchQuery))
    {
        string query = SearchQuery.Trim().ToLower();
        goals = goals.Where(g =>
            g.Title.ToLower().Contains(query) ||
            g.Description.ToLower().Contains(query)).ToList();
    }

    var activeGoals = goals.Where(g => !g.IsCompleted).ToList();

    // Оновлюємо колонку Високого пріоритету
    HighPriorityGoals.Clear();
    foreach (var goal in activeGoals.Where(g => g.Priority == PriorityLevel.High).OrderBy(g =>
g.TargetDate ?? DateTime.MaxValue))
    {
        HighPriorityGoals.Add(CreateGoalItemViewModel(goal));
    }

    // Оновлюємо колонку Середнього пріоритету
    MediumPriorityGoals.Clear();
    foreach (var goal in activeGoals.Where(g => g.Priority == PriorityLevel.Medium).OrderBy(g
=> g.TargetDate ?? DateTime.MaxValue))
    {
        MediumPriorityGoals.Add(CreateGoalItemViewModel(goal));
    }

    // Оновлюємо колонку Низького пріоритету
    LowPriorityGoals.Clear();
    foreach (var goal in activeGoals.Where(g => g.Priority == PriorityLevel.Low).OrderBy(g =>
g.TargetDate ?? DateTime.MaxValue))
    {
        LowPriorityGoals.Add(CreateGoalItemViewModel(goal));
    }

    // Оновлюємо виконані цілі (Архів)

```

```

CompletedGoals.Clear();
var completedItems = goals
    .Where(g => g.IsCompleted)
    .OrderByDescending(g => g.CompletedAt ?? g.CreatedAt);

foreach (var goal in completedItems)
{
    CompletedGoals.Add(CreateGoalItemViewModel(goal));
}
}

private GoalItemViewModel CreateGoalItemViewModel(GoalItem item)
{
    var vm = new GoalItemViewModel(item);
    vm.OnCompletionToggled += (goalId, isCompleted) =>
    {
        _goalService.ToggleGoalCompletion(_currentUser.Username, goalId, isCompleted);
        ApplyFilters(); // Оновлюємо дошки
    };
    return vm;
}

private void ExecuteAddGoal()
{
    ExecuteAddGoalWithPriority(PriorityLevel.Medium);
}

private void ExecuteAddGoalWithPriority(object? parameter)
{
    PriorityLevel priority = PriorityLevel.Medium;

    if (parameter is PriorityLevel p)
    {
        priority = p;
    }
    else if (parameter is string pStr)
    {
        priority = pStr switch
        {
            "High" or "Високий" => PriorityLevel.High,
            "Medium" or "Середній" => PriorityLevel.Medium,
            "Low" or "Низький" => PriorityLevel.Low,
            _ => PriorityLevel.Medium
        };
    }
}

```

```

        var addVm = new AddEditGoalViewModel { SelectedPriority = priority };
        addVm.OnSave += (newGoal) =>
        {
            newGoal.UserId = _currentUser.Id;
            _goalService.AddGoal(_currentUser.Username, newGoal);
            ApplyFilters();
        };

        OnOpenAddGoalDialog?.Invoke(addVm);
    }

    private void ExecuteEditGoal(object? parameter)
    {
        if (parameter is GoalItemViewModel goalVm)
        {
            var editVm = new AddEditGoalViewModel(goalVm.Model);
            editVm.OnSave += (updatedGoal) =>
            {
                _goalService.UpdateGoal(_currentUser.Username, updatedGoal);
                ApplyFilters();
            };

            OnOpenEditGoalDialog?.Invoke(editVm);
        }
    }

    private void ExecuteDeleteGoal(object? parameter)
    {
        if (parameter is GoalItemViewModel goalVm)
        {
            _goalService.DeleteGoal(_currentUser.Username, goalVm.Model.Id);
            ApplyFilters();
        }
    }

    private void ExecuteLogout()
    {
        OnLogout?.Invoke();
    }
}

public class GoalItemViewModel : ViewModelBase
{
    private readonly GoalItem _model;

```

```

        public event Action<Guid, bool>? OnCompletionToggled;

        public GoalItemViewModel(GoalItem model)
        {
            _model = model;
        }

        public GoalItem Model => _model;
        public Guid Id => _model.Id;
        public string Title => _model.Title;
        public string Description => _model.Description;
        public PriorityLevel Priority => _model.Priority;
        public string PriorityName => _model.PriorityName;
        public DateTime CreatedAt => _model.CreatedAt;
        public DateTime? TargetDate => _model.TargetDate;
        public DateTime? CompletedAt => _model.CompletedAt;

        public bool IsCompleted
        {
            get => _model.IsCompleted;
            set
            {
                if (_model.IsCompleted != value)
                {
                    _model.IsCompleted = value;
                    OnPropertyChanged();
                    OnCompletionToggled?.Invoke(_model.Id, value);
                }
            }
        }
    }
}

```

```

LoginViewModel.cs
using System;
using System.Windows.Input;
using GoalTrackerApp.Models;
using GoalTrackerApp.Services;
using Goal_Prioritizer.Models;
using Goal_Prioritizer.Services;

namespace GoalTrackerApp.ViewModels
namespace Goal_Prioritizer.ViewModels
{
    public class LoginViewModel : ViewModelBase

```

```

{
    private readonly AuthService _authService;
    private string _usernameOrEmail = string.Empty;
    private string _errorMessage = string.Empty;
    private bool _showRegisterLink = true;

    public event Action<User>? OnLoginSuccess;
    public event Action? OnOpenRegister;

    public LoginViewModel(AuthService authService)
    {
        _authService = authService;
        LoginCommand = new RelayCommand(ExecuteLogin);
        OpenRegisterCommand = new RelayCommand(ExecuteOpenRegister);
    }

    public string UsernameOrEmail
    {
        get => _usernameOrEmail;
        set => SetField(ref _usernameOrEmail, value);
    }

    public string ErrorMessage
    {
        get => _errorMessage;
        set => SetField(ref _errorMessage, value);
    }

    public bool ShowRegisterLink
    {
        get => _showRegisterLink;
        set => SetField(ref _showRegisterLink, value);
    }

    public ICommand LoginCommand { get; }
    public ICommand OpenRegisterCommand { get; }

    public void ExecuteLoginWithPassword(string password)
    {
        ErrorMessage = string.Empty;
        // reset register link visibility before attempt
        ShowRegisterLink = true;

        if (_authService.Login(UsernameOrEmail, password, out User? user, out string error))

```

```

        {
            if (user != null)
            {
                OnLoginSuccess?.Invoke(user);
            }
        }
    else
    {
        ErrorMessage = error;
        // Hide "Create account" when specific "user not found" error appears
        if (error == "Користувача з таким логіном або поштою не знайдено.")
        {
            ShowRegisterLink = false;
        }
        else
        {
            ShowRegisterLink = true;
        }
    }
}

private void ExecuteLogin(object? parameter)
{
    // У випадку якщо параметр передається з PasswordBox
    if (parameter is string password)
    {
        ExecuteLoginWithPassword(password);
    }
}

private void ExecuteOpenRegister()
{
    OnOpenRegister?.Invoke();
}
}
}

```

AddEditGoalViewModel.cs

```

using System;
using System.Collections.Generic;
using System.Windows.Input;
using GoalTrackerApp.Models;
using Goal_Prioritizer.Models;

```

```

namespace GoalTrackerApp.ViewModels

```

```

namespace Goal_Prioritizer.ViewModels
{
    public class AddEditGoalViewModel : ViewModelBase
    {
        private string _title = string.Empty;
        private string _description = string.Empty;
        private PriorityLevel _selectedPriority = PriorityLevel.Medium;
        private DateTime? _targetDate = DateTime.Now.AddDays(7);
        private string _errorMessage = string.Empty;

        public event Action<GoalItem>? OnSave;
        public event Action? OnCancel;

        public GoalItem? ExistingGoal { get; }

        public AddEditGoalViewModel(GoalItem? existingGoal = null)
        {
            ExistingGoal = existingGoal;

            if (existingGoal != null)
            {
                Title = existingGoal.Title;
                Description = existingGoal.Description;
                SelectedPriority = existingGoal.Priority;
                TargetDate = existingGoal.TargetDate;
            }

            SaveCommand = new RelayCommand(ExecuteSave);
            CancelCommand = new RelayCommand(ExecuteCancel);
        }

        public string WindowTitle => ExistingGoal == null ? "Додати нову ціль" : "Редагувати ціль";

        public string Title
        {
            get => _title;
            set => SetField(ref _title, value);
        }

        public string Description
        {
            get => _description;
            set => SetField(ref _description, value);
        }
    }
}

```

```

public PriorityLevel SelectedPriority
{
    get => _selectedPriority;
    set => SetField(ref _selectedPriority, value);
}

public DateTime? TargetDate
{
    get => _targetDate;
    set => SetField(ref _targetDate, value);
}

public string ErrorMessage
{
    get => _errorMessage;
    set => SetField(ref _errorMessage, value);
}

public IEnumerable<PriorityLevel> Priorities => new List<PriorityLevel>
{
    PriorityLevel.High,
    PriorityLevel.Medium,
    PriorityLevel.Low
};

public ICommand SaveCommand { get; }
public ICommand CancelCommand { get; }

private void ExecuteSave()
{
    ErrorMessage = string.Empty;

    if (string.IsNullOrEmpty(Title))
    {
        ErrorMessage = "Будь ласка, введіть назву цілі.";
        return;
    }

    GoalItem goal = ExistingGoal ?? new GoalItem();
    goal.Title = Title.Trim();
    goal.Description = Description?.Trim() ?? string.Empty;
    goal.Priority = SelectedPriority;
    goal.TargetDate = TargetDate;
}

```

```

        OnSave?.Invoke(goal);
    }

    private void ExecuteCancel()
    {
        OnCancel?.Invoke();
    }
}

```

PasswordHasher.cs

```

using System;
using System.Security.Cryptography;
using System.Text;

namespace GoalTrackerApp.Services
namespace Goal_Prioritizer.Services
{
    public static class PasswordHasher
    {
        public static string GenerateSalt()
        {
            byte[] saltBytes = new byte[16];
            using var rng = RandomNumberGenerator.Create();
            rng.GetBytes(saltBytes);
            return Convert.ToBase64String(saltBytes);
        }

        public static string HashPassword(string password, string salt)
        {
            if (string.IsNullOrEmpty(password)) return string.Empty;

            string saltedPassword = string.Concat(password, salt);
            byte[] bytes = Encoding.UTF8.GetBytes(saltedPassword);
            byte[] hashBytes = SHA256.HashData(bytes);

            return Convert.ToHexString(hashBytes);
        }
    }
}

```

```

        public static bool VerifyPassword(string password, string storedHash, string salt)
        {
            if (string.IsNullOrEmpty(password) || string.IsNullOrEmpty(storedHash) ||
string.IsNullOrEmpty(salt))
                return false;

            string hashOfInput = HashPassword(password, salt);
            return string.Equals(hashOfInput, storedHash, StringComparison.OrdinalIgnoreCase);
        }
    }
}

```

JsonStorageService.cs

```

using System;
using System.Collections.Generic;
using System.IO;
using System.Text.Json;
using System.Text.Encodings.Web;
using System.Text.Unicode;

namespace GoalTrackerApp.Services
namespace Goal_Prioritizer.Services
{
    public class JsonStorageService
    {
        private readonly string _dataDirectory;

        public JsonStorageService()
        {
            // Шлях до папки Data відносно папки виконання програми (.exe)
            _dataDirectory = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "Data");
            EnsureDirectoryExists(_dataDirectory);
        }

        private static void EnsureDirectoryExists(string dirPath)
        {
            if (!Directory.Exists(dirPath))
            {
                Directory.CreateDirectory(dirPath);
            }
        }

        private static JsonSerializerOptions GetJsonOptions()
        {

```

```

        return new JsonSerializerOptions
        {
            WriteIndented = true,
            Encoder = JavaScriptEncoder.Create(UnicodeRanges.All) // Підтримка кирилиці без
екранування
        };
    }

    public List<T> LoadData<T>(string relativePath)
    {
        string filePath = Path.Combine(_dataDirectory, relativePath);

        if (!File.Exists(filePath))
        {
            return new List<T>();
        }

        try
        {
            string json = File.ReadAllText(filePath);
            if (string.IsNullOrWhiteSpace(json))
            {
                return new List<T>();
            }
            return JsonSerializer.Deserialize<List<T>>(json, GetJsonOptions()) ?? new List<T>();
        }
        catch
        {
            return new List<T>();
        }
    }

    public void SaveData<T>(string relativePath, List<T> data)
    {
        string filePath = Path.Combine(_dataDirectory, relativePath);
        string? parentDir = Path.GetDirectoryName(filePath);
        if (!string.IsNullOrEmpty(parentDir))
        {
            EnsureDirectoryExists(parentDir);
        }

        string json = JsonSerializer.Serialize(data, GetJsonOptions());
        File.WriteAllText(filePath, json, System.Text.Encoding.UTF8);
    }
}

```

```

GoalService.cs
using System;
using System.Collections.Generic;
using System.IO;
using System.Linq;
using GoalTrackerApp.Models;
using Goal_Prioritizer.Models;

namespace GoalTrackerApp.Services
namespace Goal_Prioritizer.Services
{
    public class GoalService
    {
        private readonly JsonStorageService _storageService;

        public GoalService(JsonStorageService storageService)
        {
            _storageService = storageService;
        }

        private static string GetUserGoalsFilePath(string username)
        {
            // Санітизація імені папки користувача
            string safeUsername = string.Concat(username.Where(c =>
!Path.GetInvalidFileNameChars().Contains(c))).Trim();
            if (string.IsNullOrEmpty(safeUsername)) safeUsername = "default_user";

            // Внутрішній відносний шлях: Goals/{username}/goals.json
            return Path.Combine("Goals", safeUsername, "goals.json");
        }

        public List<GoalItem> GetGoalsForUser(string username)
        {
            string filePath = GetUserGoalsFilePath(username);
            return _storageService.LoadData<GoalItem>(filePath);
        }

        public void AddGoal(string username, GoalItem goal)
        {
            string filePath = GetUserGoalsFilePath(username);
            List<GoalItem> goals = _storageService.LoadData<GoalItem>(filePath);
            goals.Add(goal);
        }
    }
}

```

```

        _storageService.SaveData(filePath, goals);
    }

    public void UpdateGoal(string username, GoalItem updatedGoal)
    {
        string filePath = GetUserGoalsFilePath(username);
        List<GoalItem> goals = _storageService.LoadData<GoalItem>(filePath);
        int index = goals.FindIndex(g => g.Id == updatedGoal.Id);

        if (index != -1)
        {
            goals[index] = updatedGoal;
            _storageService.SaveData(filePath, goals);
        }
    }

    public void ToggleGoalCompletion(string username, Guid goalId, bool isCompleted)
    {
        string filePath = GetUserGoalsFilePath(username);
        List<GoalItem> goals = _storageService.LoadData<GoalItem>(filePath);
        GoalItem? goal = goals.FirstOrDefault(g => g.Id == goalId);

        if (goal != null)
        {
            goal.IsCompleted = isCompleted;
            goal.CompletedAt = isCompleted ? DateTime.Now : null;
            _storageService.SaveData(filePath, goals);
        }
    }

    public void DeleteGoal(string username, Guid goalId)
    {
        string filePath = GetUserGoalsFilePath(username);
        List<GoalItem> goals = _storageService.LoadData<GoalItem>(filePath);
        goals.RemoveAll(g => g.Id == goalId);
        _storageService.SaveData(filePath, goals);
    }
}
}

```

```

AuthService.cs
using System;
using System.Collections.Generic;
using System.Linq;

```

```

using GoalTrackerApp.Models;
using Goal_Prioritizer.Models;

namespace GoalTrackerApp.Services
namespace Goal_Prioritizer.Services
{
    public class AuthService
    {
        private const string UsersFileName = "users.json";
        private readonly JsonStorageService _storageService;

        public AuthService(JsonStorageService storageService)
        {
            _storageService = storageService;
        }

        public bool Register(string username, string email, string password, out string errorMessage)
        {
            errorMessage = string.Empty;

            if (string.IsNullOrEmpty(username))
            {
                errorMessage = "Будь ласка, введіть логін.";
                return false;
            }

            if (username.Trim().Length < 3)
            {
                errorMessage = "Логін повинен містити щонайменше 3 символи.";
                return false;
            }

            if (string.IsNullOrEmpty(email) || !email.Contains('@'))
            {
                errorMessage = "Будь ласка, введіть коректне заповнення пошти.";
                return false;
            }

            if (string.IsNullOrEmpty(password) || password.Length < 4)
            {
                errorMessage = "Пароль повинен містити щонайменше 4 символи.";
                return false;
            }
        }
    }
}

```

```

List<User> users = _storageService.LoadData<User>(UsersFileName);

if (users.Any(u => u.Username.Equals(username.Trim(), StringComparison.OrdinalIgnoreCase)))
{
    errorMessage = "Користувач із таким логіном вже існує.";
    return false;
}

if (users.Any(u => u.Email.Equals(email.Trim(), StringComparison.OrdinalIgnoreCase)))
{
    errorMessage = "Користувач із такою електронною поштою вже існує.";
    return false;
}

string salt = PasswordHasher.GenerateSalt();
string passwordHash = PasswordHasher.HashPassword(password, salt);

User newUser = new User
{
    Id = Guid.NewGuid(),
    Username = username.Trim(),
    Email = email.Trim(),
    PasswordHash = passwordHash,
    PasswordSalt = salt,
    CreatedAt = DateTime.Now
};

users.Add(newUser);
_storageService.SaveData(UsersFileName, users);

return true;
}

public bool Login(string usernameOrEmail, string password, out User? user, out string
errorMessage)
{
    user = null;
    errorMessage = string.Empty;

    if (string.IsNullOrEmpty(usernameOrEmail))
    {
        errorMessage = "Будь ласка, введіть логін або пошту.";
        return false;
    }

```

```

        if (string.IsNullOrEmpty(password))
        {
            errorMessage = "Будь ласка, введіть пароль.";
            return false;
        }

        List<User> users = _storageService.LoadData<User>(UsersFileName);

        User? foundUser = users.FirstOrDefault(u =>
            u.Username.Equals(usernameOrEmail.Trim(), StringComparison.OrdinalIgnoreCase) ||
            u.Email.Equals(usernameOrEmail.Trim(), StringComparison.OrdinalIgnoreCase));

        if (foundUser == null)
        {
            errorMessage = "Користувача з таким логіном або поштою не знайдено.";
            return false;
        }

        bool isValidPassword = PasswordHasher.VerifyPassword(password, foundUser.PasswordHash,
foundUser.PasswordSalt);
        if (!isValidPassword)
        {
            errorMessage = "Невірний пароль. Спробуйте ще раз.";
            return false;
        }

        user = foundUser;
        return true;
    }
}

```

DarkTheme.xaml

```

<ResourceDictionary xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:converters="clr-namespace:GoalTrackerApp.Converters">
    xmlns:converters="clr-namespace:Goal_Prioritizer.Converters">

    <!-- Value Converters -->

```

```

<converters:PriorityToForegroundBrushConverter x:Key="PriorityFgConverter"/>
<converters:PriorityToBackgroundBrushConverter x:Key="PriorityBgConverter"/>
<converters:PriorityToBorderBrushConverter x:Key="PriorityBorderConverter"/>
<converters:PriorityToTextConverter x:Key="PriorityTextConverter"/>
<converters:BoolToVisibilityConverter x:Key="BoolToVisConverter"/>
<converters:StringToVisibilityConverter x:Key="StringToVisConverter"/>
<converters:DateFormattingConverter x:Key="DateFormatConverter"/>

<!-- Dark Theme Color Palette -->
<SolidColorBrush x:Key="BgWindowBrush" Color="#181825"/>
<SolidColorBrush x:Key="BgCardBrush" Color="#1E1E2E"/>
<SolidColorBrush x:Key="BgInputBrush" Color="#11111B"/>
<SolidColorBrush x:Key="BgHoverBrush" Color="#313244"/>

<SolidColorBrush x:Key="BorderDarkBrush" Color="#45475A"/>
<SolidColorBrush x:Key="BorderLightBrush" Color="#585B70"/>

<SolidColorBrush x:Key="TextPrimaryBrush" Color="#CDD6F4"/>
<SolidColorBrush x:Key="TextSecondaryBrush" Color="#A6ADC8"/>
<SolidColorBrush x:Key="TextMutedBrush" Color="#6C7086"/>

<SolidColorBrush x:Key="AccentPrimaryBrush" Color="#89B4FA"/>
<SolidColorBrush x:Key="AccentHoverBrush" Color="#B4BEFE"/>
<SolidColorBrush x:Key="AccentPressedBrush" Color="#74C7EC"/>

<SolidColorBrush x:Key="DangerBrush" Color="#F38BA8"/>
<SolidColorBrush x:Key="DangerHoverBrush" Color="#E78284"/>

<!-- Base Window Style -->
<Style x:Key="DarkWindowStyle" TargetType="Window">
    <Setter Property="Background" Value="{StaticResource BgWindowBrush}"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="FontFamily" Value="Segoe UI, Segoe UI Variable, Arial"/>
    <Setter Property="FontSize" Value="14"/>
</Style>

<!-- Modern Header TextBlock -->
<Style x:Key="HeaderTextStyle" TargetType="TextBlock">
    <Setter Property="FontSize" Value="22"/>
    <Setter Property="FontWeight" Value="Bold"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="Margin" Value="0,0,0,15"/>
</Style>

```

```

<!-- Subtitle TextBlock -->
<Style x:Key="SubtitleTextStyle" TargetType="TextBlock">
    <Setter Property="FontSize" Value="13"/>
    <Setter Property="Foreground" Value="{StaticResource TextSecondaryBrush}"/>
</Style>

<!-- Primary Button Style -->
<Style x:Key="PrimaryButtonStyle" TargetType="Button">
    <Setter Property="Background" Value="{StaticResource AccentPrimaryBrush}"/>
    <Setter Property="Foreground" Value="#11111B"/>
    <Setter Property="FontWeight" Value="SemiBold"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="Padding" Value="16,10"/>
    <Setter Property="BorderThickness" Value="0"/>
    <Setter Property="Cursor" Value="Hand"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="Button">
                <Border x:Name="border"
                    Background="{TemplateBinding Background}"
                    CornerRadius="8"
                    Padding="{TemplateBinding Padding}">
                    <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsMouseOver" Value="True">
                        <Setter TargetName="border" Property="Background" Value="{StaticResource
AccentHoverBrush}"/>
                    </Trigger>
                    <Trigger Property="IsPressed" Value="True">
                        <Setter TargetName="border" Property="Background" Value="{StaticResource
AccentPressedBrush}"/>
                    </Trigger>
                    <Trigger Property="IsEnabled" Value="False">
                        <Setter TargetName="border" Property="Opacity" Value="0.5"/>
                    </Trigger>
                </ControlTemplate.Triggers>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

<!-- Secondary / Ghost Button Style -->
<Style x:Key="SecondaryButtonStyle" TargetType="Button">
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="FontWeight" Value="Medium"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="Padding" Value="14,8"/>

```

```

<Setter Property="BorderBrush" Value="{StaticResource BorderDarkBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Cursor" Value="Hand"/>
<Setter Property="Template">
    <Setter.Value>
        <ControlTemplate TargetType="Button">
            <Border x:Name="border"
                Background="{TemplateBinding Background}"
                BorderBrush="{TemplateBinding BorderBrush}"
                BorderThickness="{TemplateBinding BorderThickness}"
                CornerRadius="8"
                Padding="{TemplateBinding Padding}">
                <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>
            </Border>
            <ControlTemplate.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter TargetName="border" Property="Background" Value="{StaticResource
BgHoverBrush}"/>
                    <Setter TargetName="border" Property="BorderBrush" Value="{StaticResource
AccentPrimaryBrush}"/>
                </Trigger>
                <Trigger Property="IsPressed" Value="True">
                    <Setter TargetName="border" Property="Background" Value="{StaticResource
BgCardBrush}"/>
                </Trigger>
            </ControlTemplate.Triggers>
        </ControlTemplate>
    </Setter.Value>
</Setter>
</Style>

<!-- Danger Button Style -->
<Style x:Key="DangerButtonStyle" TargetType="Button">
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="Foreground" Value="{StaticResource DangerBrush}"/>
    <Setter Property="FontWeight" Value="Medium"/>
    <Setter Property="Padding" Value="10,6"/>
    <Setter Property="BorderBrush" Value="{StaticResource DangerBrush}"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Cursor" Value="Hand"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="Button">
                <Border x:Name="border"
                    Background="{TemplateBinding Background}"
                    BorderBrush="{TemplateBinding BorderBrush}"
                    BorderThickness="{TemplateBinding BorderThickness}"
                    CornerRadius="6"
                    Padding="{TemplateBinding Padding}">

```

```

        <ContentPresenter HorizontalAlignment="Center" VerticalAlignment="Center"/>
    </Border>
    <ControlTemplate.Triggers>
        <Trigger Property="IsMouseOver" Value="True">
            <Setter TargetName="border" Property="Background" Value="#3D1E28"/>
        </Trigger>
    </ControlTemplate.Triggers>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

<!-- Dark TextBox Style -->
<Style x:Key="DarkTextBoxStyle" TargetType="TextBox">
    <Setter Property="Background" Value="{StaticResource BgInputBrush}"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="CaretBrush" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="BorderBrush" Value="{StaticResource BorderDarkBrush}"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="10,0"/>
    <Setter Property="Height" Value="38"/>
    <Setter Property="VerticalContentAlignment" Value="Center"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="TextBox">
                <Border x:Name="border"
                    Background="{TemplateBinding Background}"
                    BorderBrush="{TemplateBinding BorderBrush}"
                    BorderThickness="{TemplateBinding BorderThickness}"
                    CornerRadius="6">
                    <ScrollViewer x:Name="PART_ContentHost"
                        Focusable="False"
                        HorizontalScrollBarVisibility="Hidden"
                        VerticalScrollBarVisibility="Hidden"
                        Margin="{TemplateBinding Padding}"
                        VerticalAlignment="{TemplateBinding VerticalContentAlignment}"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsFocused" Value="True">
                        <Setter TargetName="border" Property="BorderBrush" Value="{StaticResource
AccentPrimaryBrush}"/>
                    </Trigger>
                    <Trigger Property="IsMouseOver" Value="True">
                        <Setter TargetName="border" Property="BorderBrush" Value="{StaticResource
BorderLightBrush}"/>
                    </Trigger>
                </ControlTemplate.Triggers>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

```

```

        </Setter.Value>
    </Setter>
</Style>

<!-- Dark PasswordBox Style -->
<Style x:Key="DarkPasswordBoxStyle" TargetType="PasswordBox">
    <Setter Property="Background" Value="{StaticResource BgInputBrush}"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="CaretBrush" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="BorderBrush" Value="{StaticResource BorderDarkBrush}"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Padding" Value="10,0"/>
    <Setter Property="Height" Value="38"/>
    <Setter Property="VerticalContentAlignment" Value="Center"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="PasswordBox">
                <Border x:Name="border"
                    Background="{TemplateBinding Background}"
                    BorderBrush="{TemplateBinding BorderBrush}"
                    BorderThickness="{TemplateBinding BorderThickness}"
                    CornerRadius="6">
                    <ScrollViewer x:Name="PART_ContentHost"
                        Focusable="False"
                        HorizontalScrollBarVisibility="Hidden"
                        VerticalScrollBarVisibility="Hidden"
                        Margin="{TemplateBinding Padding}"
                        VerticalAlignment="{TemplateBinding VerticalContentAlignment}"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsFocused" Value="True">
                        <Setter TargetName="border" Property="BorderBrush" Value="{StaticResource
AccentPrimaryBrush}"/>
                    </Trigger>
                    <Trigger Property="IsMouseOver" Value="True">
                        <Setter TargetName="border" Property="BorderBrush" Value="{StaticResource
BorderLightBrush}"/>
                    </Trigger>
                </ControlTemplate.Triggers>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

<!-- Modern CheckBox Style -->
<Style x:Key="DarkCheckBoxStyle" TargetType="CheckBox">
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="FontSize" Value="14"/>

```

```

<Setter Property="Cursor" Value="Hand"/>
<Setter Property="Template">
    <Setter.Value>
        <ControlTemplate TargetType="CheckBox">
            <StackPanel Orientation="Horizontal" VerticalAlignment="Center">
                <Border x:Name="checkBorder"
                    Width="22" Height="22"
                    CornerRadius="5"
                    Background="{StaticResource BgInputBrush}"
                    BorderBrush="{StaticResource BorderDarkBrush}"
                    BorderThickness="2"
                    Margin="0,0,10,0">
                    <Path x:Name="checkMark"
                        Data="M 4,11 L 8,16 L 18,5"
                        Stroke="{StaticResource AccentPrimaryBrush}"
                        StrokeThickness="2.5"
                        StrokeLineJoin="Round"
                        StrokeStartLineCap="Round"
                        StrokeEndLineCap="Round"
                        Visibility="Collapsed"
                        HorizontalAlignment="Center"
                        VerticalAlignment="Center"/>
                </Border>
                <ContentPresenter VerticalAlignment="Center"/>
            </StackPanel>
            <ControlTemplate.Triggers>
                <Trigger Property="IsChecked" Value="True">
                    <Setter TargetName="checkMark" Property="Visibility" Value="Visible"/>
                    <Setter TargetName="checkBorder" Property="BorderBrush"
Value="{StaticResource AccentPrimaryBrush}"/>
                    <Setter TargetName="checkBorder" Property="Background" Value="#223355"/>
                </Trigger>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter TargetName="checkBorder" Property="BorderBrush"
Value="{StaticResource AccentPrimaryBrush}"/>
                </Trigger>
            </ControlTemplate.Triggers>
        </ControlTemplate>
    </Setter.Value>
</Setter>
</Style>

<!-- Priority Badge Style (Unified Layout with Custom Colors per Priority) -->
<Style x:Key="PriorityBadgeStyle" TargetType="Border">
    <Setter Property="CornerRadius" Value="6"/>
    <Setter Property="Padding" Value="10,4"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="HorizontalAlignment" Value="Left"/>
    <Setter Property="VerticalAlignment" Value="Center"/>

```

```

        <Setter Property="Background" Value="{Binding Priority, Converter={StaticResource
PriorityBgConverter}}"/>
        <Setter Property="BorderBrush" Value="{Binding Priority, Converter={StaticResource
PriorityBorderConverter}}"/>
    </Style>

    <!-- Priority Text Style -->
    <Style x:Key="PriorityTextStyle" TargetType="TextBlock">
        <Setter Property="FontSize" Value="12"/>
        <Setter Property="FontWeight" Value="Bold"/>
        <Setter Property="Foreground" Value="{Binding Priority, Converter={StaticResource
PriorityFgConverter}}"/>
    </Style>

    <!-- ComboBoxItem Style -->
    <Style x:Key="DarkComboBoxItemStyle" TargetType="ComboBoxItem">
        <Setter Property="Background" Value="Transparent"/>
        <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
        <Setter Property="Padding" Value="10,8"/>
        <Setter Property="Template">
            <Setter.Value>
                <ControlTemplate TargetType="ComboBoxItem">
                    <Border x:Name="border"
                        Background="{TemplateBinding Background}"
                        Padding="{TemplateBinding Padding}"
                        CornerRadius="4"
                        Margin="2,1">
                        <ContentPresenter/>
                    </Border>
                    <ControlTemplate.Triggers>
                        <Trigger Property="IsMouseOver" Value="True">
                            <Setter TargetName="border" Property="Background" Value="{StaticResource
BgHoverBrush}"/>
                        </Trigger>
                        <Trigger Property="IsSelected" Value="True">
                            <Setter TargetName="border" Property="Background" Value="#2A2A3C"/>
                        </Trigger>
                    </ControlTemplate.Triggers>
                </ControlTemplate>
            </Setter.Value>
        </Setter>
    </Style>

    <!-- ComboBox Toggle Button -->
    <ControlTemplate x:Key="ComboBoxToggleButton" TargetType="ToggleButton">
        <Border x:Name="templateRoot"
            Background="{TemplateBinding Background}"
            BorderBrush="{TemplateBinding BorderBrush}"

```

```

        BorderThickness="{TemplateBinding BorderThickness}"
        CornerRadius="6">
<Border x:Name="splitBorder" HorizontalAlignment="Right" Width="30">
    <Path x:Name="arrow"
        Data="M 0,0 L 4,4 L 8,0"
        HorizontalAlignment="Center"
        VerticalAlignment="Center"
        Fill="{StaticResource TextSecondaryBrush}"
        Stroke="{StaticResource TextSecondaryBrush}"
        StrokeThickness="1.5"/>
    </Border>
</Border>
</ControlTemplate>

<!-- Dark ComboBox Style -->
<Style x:Key="DarkComboBoxStyle" TargetType="ComboBox">
    <Setter Property="Background" Value="{StaticResource BgInputBrush}"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="BorderBrush" Value="{StaticResource BorderDarkBrush}"/>
    <Setter Property="BorderThickness" Value="1"/>
    <Setter Property="Height" Value="38"/>
    <Setter Property="Padding" Value="10,0"/>
    <Setter Property="FontSize" Value="14"/>
    <Setter Property="ItemContainerStyle" Value="{StaticResource DarkComboBoxItemStyle}"/>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="ComboBox">
                <Grid x:Name="mainGrid" SnapsToDevicePixels="True">
                    <ToggleButton x:Name="toggleButton"
                        Background="{TemplateBinding Background}"
                        BorderBrush="{TemplateBinding BorderBrush}"
                        BorderThickness="{TemplateBinding BorderThickness}"
                        Focusable="False"
                        IsChecked="{Binding IsDropDownOpen, Mode=TwoWay,
RelativeSource={RelativeSource TemplatedParent}}">
                        Template="{StaticResource ComboBoxToggleButton}"/>

                    <ContentPresenter x:Name="contentPresenter"
                        Content="{TemplateBinding SelectionBoxItem}"
                        ContentTemplate="{TemplateBinding SelectionBoxItemTemplate}"
                        ContentTemplateSelector="{TemplateBinding
ItemTemplateSelector}"
                        HorizontalAlignment="Left"
                        VerticalAlignment="Center"
                        Margin="{TemplateBinding Padding}"
                        IsHitTestVisible="False"/>

                    <Popup x:Name="PART_Popup"

```

```

        AllowsTransparency="True"
        IsOpen="{TemplateBinding IsDropDownOpen}"
        Placement="Bottom"
        PopupAnimation="Slide">
        <Border x:Name="dropDownBorder"
            Background="{StaticResource BgCardBrush}"
            BorderBrush="{StaticResource BorderDarkBrush}"
            BorderThickness="1"
            CornerRadius="6"
            Margin="0,4,0,0"
            MinHeight="40"
            MinWidth="{Binding ActualWidth, ElementName=mainGrid}">
            <ScrollView x:Name="DropDownScrollView" Margin="2">
                <ItemsPresenter x:Name="ItemsPresenter"
KeyboardNavigation.DirectionalNavigation="Contained"/>
            </ScrollView>
        </Border>
    </Popup>
</Grid>
</ControlTemplate>
</Setter.Value>
</Setter>
</Style>

```

```

<!-- DatePicker TextBox Style -->
<Style x:Key="DarkDatePickerTextBoxStyle" TargetType="DatePickerTextBox">
    <Setter Property="Background" Value="{StaticResource BgInputBrush}" />
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}" />
    <Setter Property="SelectionBrush" Value="{StaticResource AccentPrimaryBrush}" />
    <Setter Property="BorderThickness" Value="0" />
    <Setter Property="Padding" Value="10,0" />
    <Setter Property="VerticalContentAlignment" Value="Center" />
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="DatePickerTextBox">
                <ScrollView x:Name="PART_ContentHost"
                    Focusable="False"
                    HorizontalScrollBarVisibility="Hidden"
                    VerticalScrollBarVisibility="Hidden"
                    VerticalAlignment="Center"
                    Margin="{TemplateBinding Padding}" />
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

```

```

<!-- Dark DatePicker Style -->
<Style x:Key="DarkDatePickerStyle" TargetType="DatePicker">
    <Setter Property="Background" Value="{StaticResource BgInputBrush}" />

```

```

<Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
<Setter Property="BorderBrush" Value="{StaticResource BorderDarkBrush}"/>
<Setter Property="BorderThickness" Value="1"/>
<Setter Property="Height" Value="38"/>
<Setter Property="FontSize" Value="14"/>
<Setter Property="Template">
    <Setter.Value>
        <ControlTemplate TargetType="DatePicker">
            <Border Background="{TemplateBinding Background}"
                BorderBrush="{TemplateBinding BorderBrush}"
                BorderThickness="{TemplateBinding BorderThickness}"
                CornerRadius="6"
                Padding="2">
                <Grid x:Name="PART_Root">
                    <Grid.ColumnDefinitions>
                        <ColumnDefinition Width="*" />
                        <ColumnDefinition Width="Auto" />
                    </Grid.ColumnDefinitions>

                    <DatePickerTextBox x:Name="PART_TextBox"
                        Grid.Column="0"
                        Style="{StaticResource DarkDatePickerTextBoxStyle}"
                        HorizontalContentAlignment="Stretch"
                        VerticalContentAlignment="Center" />

                    <Button x:Name="PART_Button"
                        Grid.Column="1"
                        Content=""
                        FontSize="14"
                        Background="Transparent"
                        Foreground="{StaticResource TextSecondaryBrush}"
                        BorderThickness="0"
                        Width="30"
                        Height="30"
                        Cursor="Hand"
                        Focusable="False" />

                    <Popup x:Name="PART_Popup"
                        AllowsTransparency="True"
                        Placement="Bottom"
                        StaysOpen="False" />
                </Grid>
            </Border>
        </ControlTemplate>
    </Setter.Value>
</Setter>
</Style>

```

```

<!-- Implicit Calendar Style for dark popup -->
<Style TargetType="Calendar">
    <Setter Property="Background" Value="{StaticResource BgCardBrush}"/>
    <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
    <Setter Property="BorderBrush" Value="{StaticResource BorderDarkBrush}"/>
    <Setter Property="BorderThickness" Value="1"/>
</Style>

<!-- TabControl Dark Style -->
<Style x:Key="DarkTabControlStyle" TargetType="TabControl">
    <Setter Property="Background" Value="Transparent"/>
    <Setter Property="BorderThickness" Value="0"/>
</Style>

<Style x:Key="DarkTabItemStyle" TargetType="TabItem">
    <Setter Property="HeaderTemplate">
        <Setter.Value>
            <DataTemplate>
                <TextBlock Text="{Binding}" FontSize="15" FontWeight="SemiBold" Margin="10,6"/>
            </DataTemplate>
        </Setter.Value>
    </Setter>
    <Setter Property="Template">
        <Setter.Value>
            <ControlTemplate TargetType="TabItem">
                <Border x:Name="border"
                    Background="Transparent"
                    BorderBrush="Transparent"
                    BorderThickness="0,0,0,3"
                    Padding="16,10"
                    Margin="0,0,10,0"
                    Cursor="Hand">
                    <ContentPresenter x:Name="contentPresenter" ContentSource="Header"
Focusable="False"/>
                </Border>
                <ControlTemplate.Triggers>
                    <Trigger Property="IsSelected" Value="True">
                        <Setter TargetName="border" Property="BorderBrush" Value="{StaticResource
AccentPrimaryBrush}"/>
                        <Setter Property="Foreground" Value="{StaticResource AccentPrimaryBrush}"/>
                    </Trigger>
                    <Trigger Property="IsSelected" Value="False">
                        <Setter Property="Foreground" Value="{StaticResource TextSecondaryBrush}"/>
                    </Trigger>
                    <Trigger Property="IsMouseOver" Value="True">
                        <Setter Property="Foreground" Value="{StaticResource TextPrimaryBrush}"/>
                    </Trigger>
                </ControlTemplate.Triggers>
            </ControlTemplate>
        </Setter.Value>
    </Setter>
</Style>

```

```

        </ControlTemplate.Triggers>
    </ControlTemplate>
    </Setter.Value>
</Setter>
</Style>

</ResourceDictionary>

User.cs
using System;

namespace GoalTrackerApp.Models
namespace Goal_Prioritizer.Models
{
    public class User
    {
        public Guid Id { get; set; } = Guid.NewGuid();
        public string Username { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string PasswordHash { get; set; } = string.Empty;
        public string PasswordSalt { get; set; } = string.Empty;
        public DateTime CreatedAt { get; set; } = DateTime.Now;
    }
}

PriorityLevel.cs
namespace GoalTrackerApp.Models
namespace Goal_Prioritizer.Models
{
    public enum PriorityLevel
    {
        Low = 0,      // Низький (Жовтий)
        Medium = 1,   // Середній (Помаранчевий)
        High = 2      // Високий (Червоний)
    }

    public static class PriorityLevelExtensions
    {
        public static string ToDisplayString(this PriorityLevel priority)
        {
            return priority switch
            {
                PriorityLevel.Low => "Низький",
                PriorityLevel.Medium => "Середній",
                PriorityLevel.High => "Високий",
            }
        }
    }
}

```

```

        _ => "Низький"
    };
}
}
}

```

GoalItem.cs

```

namespace GoalTrackerApp.Models
namespace Goal_Prioritizer.Models
{
    public enum PriorityLevel
    {
        Low = 0,      // Низький (Жовтий)
        Medium = 1,   // Середній (Помаранчевий)
        High = 2      // Високий (Червоний)
    }

    public static class PriorityLevelExtensions
    {
        public static string ToDisplayString(this PriorityLevel priority)
        {
            return priority switch
            {
                PriorityLevel.Low => "Низький",
                PriorityLevel.Medium => "Середній",
                PriorityLevel.High => "Високий",
                _ => "Низький"
            };
        }
    }
}

```

ValueConverters.cs

```

using System;
using System.Globalization;
using System.Windows;
using System.Windows.Data;
using GoalTrackerApp.Models;
using Goal_Prioritizer.Models;

namespace GoalTrackerApp.Converters
namespace Goal_Prioritizer.Converters
{
    public class BoolToVisibilityConverter : IValueConverter
    {

```

```

public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
{
    if (value is bool b)
    {
        return b ? Visibility.Visible : Visibility.Collapsed;
    }
    return Visibility.Collapsed;
}

public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
{
    return value is Visibility v && v == Visibility.Visible;
}
}

public class StringToVisibilityConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        return string.IsNullOrEmpty(value as string)? Visibility.Collapsed :
Visibility.Visible;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}

public class DateFormattingConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is DateTime dt)
        {
            return dt.ToString("dd.MM.yyyy HH:mm");
        }
        return "-";
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}

```

```

public class PriorityToTextConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is PriorityLevel priority)
        {
            return priority.ToString();
        }
        return string.Empty;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}

```

PriorityToBrushConverter.cs

```

using System;
using System.Globalization;
using System.Windows.Data;
using System.Windows.Media;
using GoalTrackerApp.Models;
using Goal_Prioritizer.Models;

namespace GoalTrackerApp.Converters
namespace Goal_Prioritizer.Converters
{
    public class PriorityToForegroundBrushConverter : IValueConverter
    {
        public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
        {
            if (value is PriorityLevel priority)
            {
                return priority switch
                {
                    PriorityLevel.High => (SolidColorBrush)new
BrushConverter().ConvertFrom("#F38BA8")!, // Red
                    PriorityLevel.Medium => (SolidColorBrush)new
BrushConverter().ConvertFrom("#FAB387")!, // Orange
                    PriorityLevel.Low => (SolidColorBrush)new BrushConverter().ConvertFrom("#F9E2AF")!,
// Yellow
                    _ => (SolidColorBrush)new BrushConverter().ConvertFrom("#CDD6F4")!
                }
            }
        }
    }
}

```

```

        };
    }
    return (SolidColorBrush)new BrushConverter().ConvertFrom("#CDD6F4")!;
}

public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
{
    throw new NotImplementedException();
}
}

public class PriorityToBackgroundBrushConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is PriorityLevel priority)
        {
            return priority switch
            {
                PriorityLevel.High => (SolidColorBrush)new
BrushConverter().ConvertFrom("#3D1E28")!, // Dark Red Tint
                PriorityLevel.Medium => (SolidColorBrush)new
BrushConverter().ConvertFrom("#3D2E1E")!, // Dark Orange Tint
                PriorityLevel.Low => (SolidColorBrush)new BrushConverter().ConvertFrom("#3D391E")!,
// Dark Yellow Tint
                _ => (SolidColorBrush)new BrushConverter().ConvertFrom("#181825")!
            };
        }
        return (SolidColorBrush)new BrushConverter().ConvertFrom("#181825")!;
    }

    public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
    {
        throw new NotImplementedException();
    }
}

public class PriorityToBorderBrushConverter : IValueConverter
{
    public object Convert(object value, Type targetType, object parameter, CultureInfo culture)
    {
        if (value is PriorityLevel priority)
        {
            return priority switch
            {
                PriorityLevel.High => (SolidColorBrush)new
BrushConverter().ConvertFrom("#F38BA8")!, // Red Border

```

```

        PriorityLevel.Medium => (SolidColorBrush)new
BrushConverter().ConvertFrom("#FAB387")!, // Orange Border
        PriorityLevel.Low => (SolidColorBrush)new BrushConverter().ConvertFrom("#F9E2AF")!,
// Yellow Border
        _ => (SolidColorBrush)new BrushConverter().ConvertFrom("#45475A")!
    };
}
return (SolidColorBrush)new BrushConverter().ConvertFrom("#45475A")!;
}

public object ConvertBack(object value, Type targetType, object parameter, CultureInfo culture)
{
    throw new NotImplementedException();
}
}
}

```